Samsung PIM Simulator Summary

Intelligent System

Laboratory

Samsung PIM Simulator CMDs

❑ PIM Instruction Set Architecture

- Simulator의 PIM CMDs

Type	Command	Description	Result (DST)	Operand (SRC0)	Operand (SRC1)
Arithmetic	ADD	Addition	GRF	GRF, BANK, SRF	GRF, BANK, SRF
Arithmetic	MUL	Multiplication	GRF	GRF, BANK	GRF, BANK, SRF
Arithmetic	MAC	Multiply-Accumulate	GRF_B	GRF, BANK	GRF, BANK, SRF
Arithmetic	MAD	Multiply-Add	GRF	GRF, BANK	GRF, BANK, SRF
Data	MOV	load/store data from register to bank	GRF	GRF, SRF	GRF, BANK
Data	FILL	copy data from bank to register	GRF	GRF, BANK	GRF, BANK
Control	NOP	do nothing	-	-	-
Control	JUMP	jump instruction	-	-	-
Control	EXIT	exit instruction	-	-	-

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator CMDs

❑ PIM Instruction Set Architecture

- RISC-style 32비트 Instructions를 Support
- 3가지 Instruction 유형:
  - 4개의 Arithmetic: ADD, MUL, MAC, MAD
  - 2개의 Data Transfer: MOV, FILL
  - 3개의 Control: NOP, JUMP, EXIT
- JUMP Instruction: 0-cycle static branch
  - 미리 programming된 반복 횟수만 지원
- Operand 유형:
  - Vector Register(GRF_A, GRF_B)
  - Scalar Register(SRF)
  - Bank Row Buffer

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator CMDs

❑ PIM Instruction Set Architecture

- PIM Instructions는 CRF에 저장됨, Memory Instructions가 CRF를 trigger하여 target instruction를 수행
  - 각 Memory Instruction이 CRF PC를 increment
- DRAM Instruction이 PIM 연산을 위해 DRAM에서 data를 가져오는 위치를 결정

Samsung PIM Simulator CMDs

❑ Movement of Data

- **SB mode: standard DRAM operation**
- **HAB mode: Allowing concurrent accesses to multiple banks with a single DRAM command**
- **PIM mode: Triggers the execution of PIM instructions on the CRF by DRAM Command**

Mode	Transaction	PIM Instruction	Operation
SB	Read	-	Normal Memory Read
SB	Write	-	Normal Memory Write
HAB	Write	-	PIM Write (Host to PIM Register)
PIM	-	MOV	read or write from bank to PIM Register

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – TEST_F()

- Samsung PIM Simulator는 GoogleTest(gtest)기반의 simulation test를 수행함
- 따라서, flow를 Samsung PIM Simulator가 hbm read operation을 어떻게 test하는지 따라가며 알아본다

./sim --gtest_filter=MemBandwidthFixture.hbm_read_bandwidth

```cpp
#include <bitset>
#include <cstdint>
#include <iostream>
#include <limits>
#include <memory>
#include <random>

#include "Burst.h"
#include "MultiChannelMemorySystem.h"
#include "gtest/gtest.h"
#include "tests/PIMKernel.h"
#include "tests/TestCases.h"

using namespace std;

int main(int argc, char* argv[])
{
    if (argc > 1)
    {
        ::testing::InitGoogleTest(&argc, argv);
    }
    else
    {
        ::testing::InitGoogleTest(&argc, argv);
        testing::GTEST_FLAG(filter) = "-*bw*.*";
    }

    return RUN_ALL_TESTS();
}
```

MainTest.cpp

```cpp
TEST_F(MemBandwidthFixture, hbm_read_bandwidth)
{
    setDataSize(128 * 1024 * 64);  // in bytes
    uint64_t bw = measureCycle(false);
    float effective_bw_ratio = 0.8;
    EXPECT_TRUE(bw > 256 * effective_bw_ratio);
}
```

MemTestCases.cpp

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – SetUp()

- ./sim --gtest_filter=MemBandwidthFixture.hbm_read_bandwidth를 실행 시,
- TEST_F(MemBandwidthFixture, hbm_read_bandwidth)가 실행됨
- 이 때, TEST_F()는 MemBandwidthFixture의 SetUp()을 실행하여 mem에 대한 정보를 initialize함

MemTestCases.cpp

```
1. TEST_F(MemBandwidthFixture, hbm_read_bandwidth)
2. {
3.     setDataSize(128 * 1024 * 64);  // in bytes
4.     uint64_t bw = measureCycle(false);
5.     float effective_bw_ratio = 0.8;
6.     EXPECT_TRUE(bw > 256 * effective_bw_ratio);
7. }
```

TestCases.h

```
1.     virtual void SetUp()
2.     {
3.         cur_cycle = 0;
4.         mem = make_shared<MultiChannelMemorySystem>("ini/HBM2_samsung_2M_16B_x64.ini",
5.                                                     "system_hbm.ini", ".", "example_app", 256 * 16);
6.         mem_size = (uint64_t)getConfigParam(UINT, "NUM_CHANS") * getConfigParam(UINT, "NUM_RANKS") *
7.                    getConfigParam(UINT, "NUM_BANK_GROUPS") * getConfigParam(UINT, "NUM_BANKS") *
8.                    getConfigParam(UINT, "NUM_ROWS") * getConfigParam(UINT, "NUM_COLS");
9.         basic_stride =
10.            (getConfigParam(UINT, "JEDEC_DATA_BUS_BITS") * getConfigParam(UINT, "BL") / 8);
11.    }
```

```
1. MultiChannelMemorySystem::MultiChannelMemorySystem(const string& deviceIniFilename_,
2.                                                    const string& systemIniFilename_,
3.                                                    const string& pwd_, const string& traceFilename_,
4.                                                    unsigned megsOfMemory_, string* visFilename_)
5.     : megsOfMemory(megsOfMemory_),
6.       deviceIniFilename(deviceIniFilename_),
7.       systemIniFilename(systemIniFilename_),
8.       traceFilename(traceFilename_),
9.       pwd(pwd_),
10.      visFilename(visFilename_),
11.      clockDomainCrosser(new ClockDomain::Callback<MultiChannelMemorySystem, void>(
12.          this, &MultiChannelMemorySystem::actual_update)),
13.      csvOut(new CSVWriter(visDataOut))
14. {
15.    currentClockCycle = 0;
16.    if (visFilename)
17.        printf("CC VISFILENAME=%s\n", visFilename->c_str());
18.
19.    if (!isPowerOfTwo(megsOfMemory))
20.    {
21.        ERROR("Please specify a power of 2 memory size");
22.        abort();
23.    }
24.
25.    ConfigurationDB& configDB = ConfigurationDB::getDB();
26.    configDB.initialize();
27.    configDB.updatefromFile(deviceIniFilename);
28.    configDB.updatefromFile(systemIniFilename);
29.
30.    addrMapping = new AddrMapping();
31.    configuration = new Configuration(*addrMapping);
32.    numFence = new unsigned[configuration->NUM_CHANS]();
33.
34.    for (size_t i = 0; i < configuration->NUM_CHANS; i++)
35.    {
36.        MemorySystem* channel = new MemorySystem(i, megsOfMemory / configuration->NUM_CHANS,
37.                                                 (*csvOut), dramsimLog, *configuration);
38.        channels.push_back(channel);
39.    }
40. }
```

MultiChannelMemorySystem.cpp

- callback 등록
- Clock 초기화
- 각 .ini의 내용을 configDB에 저장
- 각 .ini의 내용을 기반해 address bit 정보 가져옴
- Channel 개수만큼 MemorySystem 생성

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – SetUp()

- **Memory 생성 시**
  - **Channel개수만큼 MemorySystem 생성 → 각 MemorySystem 마다 Rank개수만큼 Rank Class 생성**

```cpp
MultiChannelMemorySystem::MultiChannelMemorySystem(const string& deviceIniFilename_,
                                                   const string& systemIniFilename_,
                                                   const string& pwd_, const string& traceFilename_,
                                                   unsigned megsOfMemory_, string* visFilename_)
    : megsOfMemory(megsOfMemory_),
      deviceIniFilename(deviceIniFilename_),
      systemIniFilename(systemIniFilename_),
      traceFilename(traceFilename_),
      pwd(pwd_),
      visFilename(visFilename_),
      clockDomainCrosser(new ClockDomain::Callback<MultiChannelMemorySystem, void>(
          this, &MultiChannelMemorySystem::actual_update)),
      csvOut(new CSVWriter(visDataOut))
{
    currentClockCycle = 0;
    if (visFilename)
        printf("CC VISFILENAME=%s\n", visFilename->c_str());

    if (!isPowerOfTwo(megsOfMemory))
    {
        ERROR("Please specify a power of 2 memory size");
        abort();
    }

    ConfigurationDB& configDB = ConfigurationDB::getDB();
    configDB.initialize();
    configDB.updatefromFile(deviceIniFilename);
    configDB.updatefromFile(systemIniFilename);

    addrMapping = new AddrMapping();
    configuration = new Configuration(*addrMapping);
    numFence = new unsigned[configuration->NUM_CHANS]();

    for (size_t i = 0; i < configuration->NUM_CHANS; i++)
    {
        MemorySystem* channel = new MemorySystem(i, megsOfMemory / configuration->NUM_CHANS,
                                                 (*csvOut), dramsimLog, *configuration);
        channels.push_back(channel);
    }
}
```

Channel 개수만큼
MemorySystem 생성

MultiChannelMemorySystem.cpp

```cpp
MemorySystem::MemorySystem(unsigned id, unsigned int megsOfMemory, CSVWriter& csvOut_,
                           ostream& simLog, Configuration& configuration)
    : dramsimLog(simLog),
      ReturnReadData(NULL),
      WriteDataDone(NULL),
      systemID(id),
      csvOut(csvOut_),
      numOnTheFlyTransactions(0),
      config(configuration)
{
    currentClockCycle = 0;

    DEBUG("===== MemorySystem " << systemID << " =====");
    uint64_t bytePerRank = static_cast<uint64_t>(
        (getConfigParam(UINT64, "NUM_ROWS") *
         (getConfigParam(UINT, "NUM_COLS") * getConfigParam(UINT, "DEVICE_WIDTH")) *
         getConfigParam(UINT, "NUM_BANKS") *
         (getConfigParam(UINT64, "JEDEC_DATA_BUS_BITS") / getConfigParam(UINT, "DEVICE_WIDTH"))) /
        8);
    uint64_t megsOfStoragePerRank = bytePerRank >> 20;  // bytes to MB (2^20 = 1,048,576 = Mega)

    num_ranks_ = (megsOfMemory / Byte2MB(bytePerRank));

    // If this is set, effectively override the number of ranks
    if (megsOfMemory != 0)
    {
        num_ranks_ = (num_ranks_ > 0) ? num_ranks_ : 1;
        if (num_ranks_ == 0)
        {
            PRINT("WARNING: Cannot create memory system with "
                  << megsOfMemory << "MB, defaulting to minimum size of " << megsOfStoragePerRank
                  << "MB");
        }
        setDevConfigParam(UINT, "NUM_RANKS", num_ranks_);
        // FIXME, need to synchronize between ConfigurationDB and Configuration
        config.NUM_RANKS = num_ranks_;
    }

    unsigned num_devices =
        (getConfigParam(UINT, "JEDEC_DATA_BUS_BITS") / getConfigParam(UINT, "DEVICE_WIDTH"));
    setDevConfigParam(UINT, "NUM_DEVICES", num_devices);
    setDevConfigParam(UINT64, "TOTAL_STORAGE", num_ranks_ * Byte2MB(bytePerRank));

    PRINT("CH. " << systemID << " TOTAL_STORAGE : " << getConfigParam(UINT64, "TOTAL_STORAGE")
                 << "MB | " << getConfigParam(UINT, "NUM_RANKS") << " Ranks | "
                 << getConfigParam(UINT, "NUM_DEVICES") << " Devices per rank");

    memoryController = new MemoryController(this, csvOut, dramsimLog, config);

    // TODO: change to other vector constructor?
    ranks = new vector<Rank*>();

    for (size_t i = 0; i < num_ranks_; i++)
    {
        Rank* r = new Rank(dramsimLog, config);
        r->setChanId(systemID);
        r->setRankId(i);
        r->attachMemoryController(memoryController);
        r->pimRank->setChanId(systemID);
        r->pimRank->setRankId(i);
        ranks->push_back(r);
    }

    memoryController->attachRanks(ranks);
}
```

MemorySystem.cpp

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – SetUp()

- **Memory 생성 시**
  - **Rank개수만큼 Rank Class 생성 → Rank는 자신의 Class 변수에 channel, rank id 및 Bank들 관련 변수 초기화**

```cpp
MemorySystem::MemorySystem(unsigned id, unsigned int megsOfMemory, CSVWriter& csvOut_,
                           ostream& simLog, Configuration& configuration)
    : dramsimLog(simLog),
      ReturnReadData(NULL),
      WriteDataDone(NULL),
      systemID(id),
      csvOut(csvOut_),
      numOnTheFlyTransactions(0),
      config(configuration)
{
    currentClockCycle = 0;

    DEBUG("===== MemorySystem " << systemID << " =====");
    uint64_t bytePerRank = static_cast<uint64_t>(
        (getConfigParam(UINT64, "NUM_ROWS") *
         (getConfigParam(UINT, "NUM_COLS") * getConfigParam(UINT, "DEVICE_WIDTH")) *
         getConfigParam(UINT, "NUM_BANKS") *
         (getConfigParam(UINT64, "JEDEC_DATA_BUS_BITS") / getConfigParam(UINT, "DEVICE_WIDTH"))) /
        8);
    uint64_t megsOfStoragePerRank = bytePerRank >> 20;  // bytes to MB (2^20 = 1,048,576 = Mega)

    num_ranks_ = (megsOfMemory / Byte2MB(bytePerRank));

    // If this is set, effectively override the number of ranks
    if (megsOfMemory != 0)
    {
        num_ranks_ = (num_ranks_ > 0) ? num_ranks_ : 1;
        if (num_ranks_ == 0)
        {
            PRINT("WARNING: Cannot create memory system with "
                  << megsOfMemory << "MB, defaulting to minimum size of " << megsOfStoragePerRank
                  << "MB");
        }
        setDevConfigParam(UINT, "NUM_RANKS", num_ranks_);
        // FIXME, need to synchronize between ConfigurationDB and Configuration
        config.NUM_RANKS = num_ranks_;
    }

    unsigned num_devices =
        (getConfigParam(UINT, "JEDEC_DATA_BUS_BITS") / getConfigParam(UINT, "DEVICE_WIDTH"));
    setDevConfigParam(UINT, "NUM_DEVICES", num_devices);
    setDevConfigParam(UINT64, "TOTAL_STORAGE", num_ranks_ * Byte2MB(bytePerRank));

    PRINT("CH. " << systemID << " TOTAL_STORAGE : " << getConfigParam(UINT64, "TOTAL_STORAGE")
                 << "MB | " << getConfigParam(UINT, "NUM_RANKS") << " Ranks | "
                 << getConfigParam(UINT, "NUM_DEVICES") << " Devices per rank");

    memoryController = new MemoryController(this, csvOut, dramsimLog, config);

    // TODO: change to other vector constructor?
    ranks = new vector<Rank*>();

    for (size_t i = 0; i < num_ranks_; i++)
    {
        Rank* r = new Rank(dramsimLog, config);
        r->setChanId(systemID);
        r->setRankId(i);
        r->attachMemoryController(memoryController);
        r->pimRank->setChanId(systemID);
        r->pimRank->setRankId(i);
        ranks->push_back(r);
    }

    memoryController->attachRanks(ranks);
}
```

MemorySystem.cpp

```cpp
Rank::Rank(ostream& simLog, Configuration& configuration)
    : chanId(-1),
      rankId(-1),
      dramsimLog(simLog),
      isPowerDown(false),
      refreshWaiting(false),
      readReturnCountdown(0),
      banks(getConfigParam(UINT, "NUM_BANKS"), Bank(simLog)),
      bankStates(getConfigParam(UINT, "NUM_BANKS"), BankState(simLog)),
      config(configuration),
      outgoingDataPacket(NULL),
      dataCyclesLeft(0),
      mode_(dramMode::SB)
{
    memoryController = NULL;
    currentClockCycle = 0;
    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = sbmr1_ = sbmr2_ = false;

    pimRank = make_shared<PIMRank>(dramsimLog, config);
    pimRank->attachRank(this);
}
```

Rank.cpp

```cpp
class Rank : public SimulatorObject
{
  private:
    int chanId;
    int rankId;
    ostream& dramsimLog;
    bool isPowerDown;
    Configuration& config;

  public:
    // functions
    Rank(ostream& simLog, Configuration& configuration);
    virtual ~Rank();

    void receiveFromBus(BusPacket* packet);
    void check(BusPacket* packet);
    void updateState(BusPacket* packet);
    void sendToBank(BusPacket* packet);

    void checkBank(BusPacketType type, int bank, int row);
    void updateBank(BusPacketType type, int bank, int row, bool targetBank, bool targetBankgroup);
    void attachMemoryController(MemoryController* mc);
    int getChanId() const;
    void setChanId(int id);
    int getRankId() const;
    void setRankId(int id);
    void update();
    void powerUp();
    void powerDown();

    void readSb(BusPacket* packet);
    void writeSb(BusPacket* packet);

    // fields
    MemoryController* memoryController;
    BusPacket* outgoingDataPacket;
    shared_ptr<PIMRank> pimRank;
    unsigned dataCyclesLeft;
    bool refreshWaiting;

    // these are vectors so that each element is per-bank
    vector<BusPacket*> readReturnPacket;
    vector<unsigned> readReturnCountdown;

    vector<Bank> banks;
    vector<BankState> bankStates;

    dramMode mode_;
    bool abmr1Even_, abmr1Odd_, abmr2Even_, abmr2Odd_, sbmr1_, sbmr2_;

    const char* getModeColor()
    {
        switch (mode_)
        {
            case dramMode::SB:
                return END;
            case dramMode::HAB:
                return GREEN;
            case dramMode::HAB_PIM:
                return CYAN;
        }
        return GRAY;
    }
};
```

Rank.h

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- mem 초기화 이후 data_size_in_byte를 설정
- measureCycle()을 통하여 성능을 simulate
- generateMemTraffic()은 memory transaction을 생성하는 함수 → Memory Trace를 생성하는 역할과 동일

MemTestCases.cpp

```
1. TEST_F(MemBandwidthFixture, hbm_read_bandwidth)
2. {
3.     setDataSize(128 * 1024 * 64);  // in bytes
4.     uint64_t bw = measureCycle(false);
5.     float effective_bw_ratio = 0.8;
6.     EXPECT_TRUE(bw > 256 * effective_bw_ratio);
7. }
```

TestCases.h

```
1.     void setDataSize(unsigned size)
2.     {
3.         data_size_in_byte = size;
4.     }
```

TestCases.h

```
1.     uint64_t measureCycle(bool is_write)
2.     {
3.         printTestMessage();
4.         write_ = is_write;
5.         generateMemTraffic(is_write);
6.
7.         while (mem->hasPendingTransactions())
8.         {
9.             cur_cycle++;
10.            mem->update();
11.        }
12.
13.        return cur_cycle;
14.    }
```

TestCases.h

```
1.     void printTestMessage()
2.     {
3.         cout << ">> Bandwidth Test" << endl;
4.         cout << "  Num. channels: " << getConfigParam(UINT, "NUM_CHANS") << endl;
5.         cout << "  Num. ranks: " << getConfigParam(UINT, "NUM_RANKS") << endl;
6.         cout << "  Num. banks: " << getConfigParam(UINT, "NUM_BANKS") << endl;
7.         cout << "  Data size (byte): " << data_size_in_byte << endl;
8.     }
```

TestCases.h

```
1.     void generateMemTraffic(bool is_write)
2.     {
3.         int num_trans = 0;
4.         BurstType nullBst;
5.
6.         for (uint64_t i = 0; i < mem_size; ++i)
7.         {
8.             if (num_trans >= (data_size_in_byte / basic_stride))
9.             {
10.                break;
11.            }
12.            uint64_t addr = i * basic_stride;
13.            mem->addTransaction(is_write, addr, &nullBst);
14.            num_trans++;
15.        }
16.    }
```

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle() → generateMemTraffic()

- generateMemTraffic()은 어떤, 얼만큼의 memory transaction(Read or Write)을 수행할 것인지 결정하여 transactionQueue를 채움
- transactionQueue는 최종적으로 MemoryController.cpp에서 관리하며, scheduling logic에 따라 issue됨
- MultiChannelMemorySystem::findChannelNumber()은 다음 page에서 설명
- mem_size 및 basic_stride는 PPT page7의 SetUp()에서 정의됨
  - mem_size = NUM_CHANS × NUM_RANKS × NUM_BANK_GROUPS × NUM_BANKS × NUM_ROWS × NUM_COLS
  - basic_stride = (JEDEC_DATA_BUS_BITS × BL) / 8 [ex. JEDEC_DATA_BUS_BITS = 64, BL = 4, basic_stride = (64 bits × 4) / 8 bytes/bit = 32 bytes (= 한 번의 READ/WRITE transaction 시 전송되는 data 크기)]

TestCases.h

```
1.  void generateMemTraffic(bool is_write)
2.  {
3.      int num_trans = 0;
4.      BurstType nullBst;
5.
6.      for (uint64_t i = 0; i < mem_size; ++i)
7.      {
8.          if (num_trans >= (data_size_in_byte / basic_stride))
9.          {
10.             break;
11.         }
12.         uint64_t addr = i * basic_stride;
13.         mem->addTransaction(is_write, addr, &nullBst);
14.         num_trans++;
15.     }
16. }
```

MultiChannelMemorySystem.cpp

```
1. bool MultiChannelMemorySystem::addTransaction(bool isWrite, uint64_t addr, BurstType* data)
2. {
3.     unsigned channelNumber = findChannelNumber(addr);
4.     return channels[channelNumber]->addTransaction(isWrite, addr, data);
5. }
```

```
1. unsigned MultiChannelMemorySystem::findChannelNumber(uint64_t addr)
2. {
3.     // Single channel case is a trivial shortcut case
4.     if (configuration->NUM_CHANS == 1)
5.     {
6.         return 0;
7.     }
8.
9.     if (!isPowerOfTwo(configuration->NUM_CHANS))
10.    {
11.        ERROR("We can only support power of two # of channels.\n"
12.              << "I don't know what Intel was thinking, but trying to address map half"
13.                 " a bit is a neat trick that we're not sure how to do");
14.        abort();
15.    }
16.
17.    // only chan is used from this set
18.    unsigned channelNumber, rank, bank, row, col;
19.    addrMapping->addressMapping(addr, channelNumber, rank, bank, row, col);
20.    if (channelNumber >= configuration->NUM_CHANS)
21.    {
22.        ERROR("Got channel index " << channelNumber << " but only " << configuration->NUM_CHANS
23.                                   << " exist");
24.        abort();
25.    }
26.    return channelNumber;
27. }
```

MultiChannelMemorySystem.cpp

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle() → generateMemTraffic()

- addressMapping의 경우 주어진 address를 shifting 및 slicing함으로써 처리됨
- addressMapping에서 address의 하위 6bit(= transaction size 및 burst size 나타내는 bit)는 shift right하여 제거해야 함
  - Burst: 한 번에 읽는 작은 데이터 덩어리 (DDR에서 보통 8byte), Transaction: 전체 요청 단위, 여러 burst로 구성, 보통 cache line 크기와 동일(ex. 64byte)
  - Controller는 Transaction 1개를 받아서 처리 & Column Address는 Burst 중 Byte마다 +1 증가 → 따라서, if transaction = 64byte이면, 64byte = 2^6 byte → 6 bit right shift 필요
- Shift하고 남은 bit들은 적절히 slicing하여 scheme에 따라, (channel : rank : bank : row : col)로 분리 (scheme마다 slice방법 다름)
- MultiChannelMemorySystem::findChannelNumber()는 channelNumber(= channel id)를 반환

MultiChannelMemorySystem.cpp

```
unsigned MultiChannelMemorySystem::findChannelNumber(uint64_t addr)
{
    // Single channel case is a trivial shortcut case
    if (configuration->NUM_CHANS == 1)
    {
        return 0;
    }

    if (!isPowerOfTwo(configuration->NUM_CHANS))
    {
        ERROR("We can only support power of two # of channels.\n"
              << "I don't know what Intel was thinking, but trying to address map half"
                 " a bit is a neat trick that we're not sure how to do");
        abort();
    }

    // only chan is used from this set
    unsigned channelNumber, rank, bank, row, col;
    addrMapping->addressMapping(addr, channelNumber, rank, bank, row, col);
    if (channelNumber >= configuration->NUM_CHANS)
    {
        ERROR("Got channel index " << channelNumber << " but only " << configuration->NUM_CHANS
                                   << " exist");
        abort();
    }
    return channelNumber;
}
```

addressMapping.cpp

```
void AddrMapping::addressMapping(uint64_t physicalAddress, unsigned& newTransactionChan,
                                 unsigned& newTransactionRank, unsigned& newTransactionBank,
                                 unsigned& newTransactionRow, unsigned& newTransactionColumn)
{
    if ((physicalAddress & transactionMask) != 0)
    {
        DEBUG("WARNING: address 0x" << std::hex << physicalAddress << std::dec
                                    << " is not aligned to the request size of "
                                    << transactionSize);
    }
    // each burst will contain JEDEC_DATA_BUS_BITS/8 bytes of data, so the bottom
    // bits (3 bits for a single channel DDR system) are thrown away before mapping the other bits
    physicalAddress >>= byteOffsetWidth;

    // The next thing we have to consider is that when a request is made for a
    // we've taken into account the granularity of a single burst by shifting
    // off the bottom 3 bits, but a transaction has to take into account the
    // burst length (i.e. the requests will be aligned to cache line sizes which
    // should be equal to transactionSize above).
    //
    // Since the column address increments internally on bursts, the bottom n
    // bits of the column (colLow) have to be zero in order to account for the
    // total size of the transaction. These n bits should be shifted off the
    // address and also subtracted from the total column width.
    //
    // I am having a hard time explaining the reasoning here, but it comes down
    // this: for a 64 byte transaction, the bottom 6 bits of the address must be
    // zero. These zero bits must be made up of the byte offset (3 bits) and
    // also
    // from the bottom bits of the column
    //
    // For example: cowLowBits = log2(64bytes) - 3 bits = 3 bits

    physicalAddress >>= colLowBitWidth;

    if (DEBUG_ADDR_MAP)
    {
        DEBUG("Bit widths: ch:" << channelBitWidth << " r:" << rankBitWidth << " b:" << bankBitWidth
                                << " row:" << rowBitWidth << " colLow:" << colLowBitWidth
                                << " colHigh:" << colHighBitWidth << " off:" << byteOffsetWidth
                                << " Total:"
                                << (channelBitWidth + rankBitWidth + bankBitWidth + rowBitWidth +
                                    colLowBitWidth + colHighBitWidth + byteOffsetWidth));
    }

    // perform various address mapping schemes
    if (addressMappingScheme == Scheme1)
    {
        // chan:rank:row:col:bank
        newTransactionBank = diffBitWidth(&physicalAddress, bankBitWidth);
        newTransactionColumn = diffBitWidth(&physicalAddress, colHighBitWidth);
        newTransactionRow = diffBitWidth(&physicalAddress, rowBitWidth);
        newTransactionRank = diffBitWidth(&physicalAddress, rankBitWidth);
        newTransactionChan = diffBitWidth(&physicalAddress, channelBitWidth);
    }
    else if (addressMappingScheme == Scheme2)
    {
        // chan:row:col:bank:rank
        newTransactionRank = diffBitWidth(&physicalAddress, rankBitWidth);
        newTransactionBank = diffBitWidth(&physicalAddress, bankBitWidth);
        newTransactionColumn = diffBitWidth(&physicalAddress, colHighBitWidth);
        newTransactionRow = diffBitWidth(&physicalAddress, rowBitWidth);
        newTransactionChan = diffBitWidth(&physicalAddress, channelBitWidth);
    }
    else if (addressMappingScheme == Scheme3)
    {
        // chan:rank:bank:col:row
        newTransactionRow = diffBitWidth(&physicalAddress, rowBitWidth);
        newTransactionColumn = diffBitWidth(&physicalAddress, colHighBitWidth);
        newTransactionBank = diffBitWidth(&physicalAddress, bankBitWidth);
        newTransactionRank = diffBitWidth(&physicalAddress, rankBitWidth);
        newTransactionChan = diffBitWidth(&physicalAddress, channelBitWidth);
    }
    else if (addressMappingScheme == Scheme4)
    {
        // chan:rank:bank:row:col
        newTransactionColumn = diffBitWidth(&physicalAddress, colHighBitWidth);
        newTransactionRow = diffBitWidth(&physicalAddress, rowBitWidth);
        newTransactionBank = diffBitWidth(&physicalAddress, bankBitWidth);
        newTransactionRank = diffBitWidth(&physicalAddress, rankBitWidth);
        newTransactionChan = diffBitWidth(&physicalAddress, channelBitWidth);
    }
    else if (addressMappingScheme == Scheme5)
    {
        // chan:row:col:rank:bank
        newTransactionBank = diffBitWidth(&physicalAddress, bankBitWidth);
        newTransactionRank = diffBitWidth(&physicalAddress, rankBitWidth);
        newTransactionColumn = diffBitWidth(&physicalAddress, colHighBitWidth);
        newTransactionRow = diffBitWidth(&physicalAddress, rowBitWidth);
        newTransactionChan = diffBitWidth(&physicalAddress, channelBitWidth);
    }
    else if (addressMappingScheme == Scheme6)
    {
        // chan:row:bank:rank:col
        newTransactionColumn = diffBitWidth(&physicalAddress, colHighBitWidth);
        newTransactionRank = diffBitWidth(&physicalAddress, rankBitWidth);
        newTransactionBank = diffBitWidth(&physicalAddress, bankBitWidth);
        newTransactionRow = diffBitWidth(&physicalAddress, rowBitWidth);
        newTransactionChan = diffBitWidth(&physicalAddress, channelBitWidth);
    }
    // clone of scheme 5, but channel moved to lower bits
    else if (addressMappingScheme == Scheme7)
    {
        // row:col:rank:bank:chan
        newTransactionChan = diffBitWidth(&physicalAddress, channelBitWidth);
        newTransactionBank = diffBitWidth(&physicalAddress, bankBitWidth);
        newTransactionRank = diffBitWidth(&physicalAddress, rankBitWidth);
        newTransactionColumn = diffBitWidth(&physicalAddress, colHighBitWidth);
        newTransactionRow = diffBitWidth(&physicalAddress, rowBitWidth);
    }
    else if (addressMappingScheme == Scheme8)
    {
        // rank:row:col:bg:bank:chan
        newTransactionChan = diffBitWidth(&physicalAddress, channelBitWidth);
        newTransactionBank = diffBitWidth(&physicalAddress, bankBitWidth - bankgroupBitWidth);
        newTransactionBank |= diffBitWidth(&physicalAddress, bankgroupBitWidth)
                              << (bankBitWidth - bankgroupBitWidth);
        newTransactionColumn = diffBitWidth(&physicalAddress, colHighBitWidth);
        newTransactionRow = diffBitWidth(&physicalAddress, rowBitWidth);
        newTransactionRank = diffBitWidth(&physicalAddress, rankBitWidth);
    }

    else
    {
        ERROR("== Error - Unknown Address Mapping Scheme");
        exit(-1);
    }

    if (DEBUG_ADDR_MAP)
    {
        DEBUG("Mapped Ch=" << newTransactionChan << " Rank=" << newTransactionRank
                           << " Bank=" << newTransactionBank << " Row=" << newTransactionRow
                           << " Col=" << newTransactionColumn << "\n");
    }
}
```

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle() → generateMemTraffic()

- addTransaction()은 해당 transaction의 addr가 나타내는 channel의 Memory Controller에게 해당 transaction을 넘김
- Memory Controller는 transactionQueue가 꽉 차지 않았으면, 해당 transaction을 Queue에 저장
- 꽉 찼으면 pendingTransactions 배열에 저장됨
- trans->timeAdded는 기존 DRAMSim2에서 latency 지표 계산에 쓰임
  - But, DRAMSim2에서만 trans->timeAdded가 statistic에 반영되고, PIMSimulator는 이를 반영하지 않음 (사실상 안 쓰이는 변수)

MultiChannelMemorySystem.cpp

```
1. bool MultiChannelMemorySystem::addTransaction(bool isWrite, uint64_t addr, BurstType* data)
2. {
3.     unsigned channelNumber = findChannelNumber(addr);
4.     return channels[channelNumber]->addTransaction(isWrite, addr, data);
5. }
```

MemorySystem.cpp

```
1. bool MemorySystem::addTransaction(bool isWrite, uint64_t addr, BurstType* data)
2. {
3.     TransactionType type = isWrite ? DATA_WRITE : DATA_READ;
4.     Transaction* trans = new Transaction(type, addr, data);
5.     return addTransaction(trans);
6. }
```

MemorySystem.cpp

```
1. bool MemorySystem::addTransaction(Transaction* trans)
2. {
3.     if (memoryController->WillAcceptTransaction())
4.     {
5.         return memoryController->addTransaction(trans);
6.     }
7.     else
8.     {
9.         pendingTransactions.push_back(trans);
10.        return true;
11.    }
12. }
```

MemoryController.cpp

```
1. bool MemoryController::WillAcceptTransaction()
2. {
3.     return transactionQueue.size() < getConfigParam(UINT, "TRANS_QUEUE_DEPTH");
4. }
```

MemoryController.cpp

```
1. bool MemoryController::addTransaction(Transaction* trans)
2. {
3.     if (WillAcceptTransaction())
4.     {
5.         parentMemorySystem->numOnTheFlyTransactions++;
6.         trans->timeAdded = currentClockCycle;
7.         transactionQueue.push_back(trans);
8.         return true;
9.     }
10.    else
11.    {
12.        return false;
13.    }
14. }
```

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle() → update()

- measureCycle()을 통하여 성능을 simulate
- CPU clock과 DRAM clock의 clock domain을 정렬하며, callback을 호출
- 등록된 callback은 actual_update() 임 (PPT page7 참고)

```
1. void MultiChannelMemorySystem::setCPUClockSpeed(uint64_t cpuClkFreqHz)
2. {
3.     uint64_t dramsimClkFreqHz = (uint64_t)(1.0 / (configuration->tCK * 1e-9));
4.     clockDomainCrosser.clock1 = dramsimClkFreqHz;  // ← DRAM 클록 (Hz)
5.     clockDomainCrosser.clock2 = cpuClkFreqHz;      // ← CPU 클록 (Hz)
6. }
```

```
1. namespace ClockDomain
2. {
3. // "Default" crosser with a 1:1 ratio
4. ClockDomainCrosser::ClockDomainCrosser(ClockUpdateCB* _callback)
5.     : callback(_callback), clock1(1UL), clock2(1UL), counter1(0UL), counter2(0UL)
6.
```

TestCases.h

```
1.    uint64_t measureCycle(bool is_write)
2.    {
3.        printTestMessage();
4.        write_ = is_write;
5.        generateMemTraffic(is_write);
6.
7.        while (mem->hasPendingTransactions())
8.        {
9.            cur_cycle++;
10.           mem->update();
11.       }
12.
13.       return cur_cycle;
14.   }
```

MultiChannelMemorySystem.cpp

```
1. void MultiChannelMemorySystem::update()
2. {
3.     clockDomainCrosser.update();
4. }
5.
6. void MultiChannelMemorySystem::actual_update()
7. {
8.     if (currentClockCycle == 0)
9.     {
10.        InitOutputFiles(traceFilename);
11.        DEBUG("DRAMSim2 Clock Frequency =" << clockDomainCrosser.clock1
12.                                           << "Hz, CPU Clock Frequency="
13.                                           << clockDomainCrosser.clock2 << "Hz");
14.    }
15.
16.    if (currentClockCycle > 0 && currentClockCycle % configuration->EPOCH_LENGTH == 0)
17.    {
18.        (*csvOut) << "ms" << currentClockCycle * configuration->tCK * 1E-6;
19.        for (size_t i = 0; i < configuration->NUM_CHANS; i++)
20.        {
21.            channels[i]->printStats(false);
22.        }
23.        csvOut->finalize();
24.    }
25.
26.    for (size_t i = 0; i < configuration->NUM_CHANS; i++)
27.    {
28.        channels[i]->update();
29.    }
30.
31.    currentClockCycle++;
32. }
```

ClockDomain.cpp

```
1. void ClockDomainCrosser::update()
2. {
3.     // short circuit case for 1:1 ratios
4.     if (clock1 == clock2 && callback)
5.     {
6.         (*callback)();
7.         return;
8.     }
9.
10.    // Update counter 1.
11.    counter1 += clock1;
12.
13.    while (counter2 < counter1)
14.    {
15.        counter2 += clock2;
16.        // PRINT("CALLBACK: counter1= " << counter1 << "; counter2= " << counter2/ << "; ");
17.        // PRINT("callback address: " << (uint64_t)callback);
18.        if (callback)
19.        {
20.            // PRINT("Callback() " << (uint64_t)callback<< "Counters:1="<<counter1<<",
21.            // 2="<<counter2);
22.            (*callback)();
23.        }
24.    }
25.
26.    if (counter1 == counter2)
27.    {
28.        counter1 = 0;
29.        counter2 = 0;
30.    }
31. }
```

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → MultiChannelMemorySystem::actual_update() → MemorySystem::update() → Rank::update()
- Rank::update() – 특이사항: 현재 Simulator는 독립적인 command/read/write bus를 사용하고 있음
  1. Read Latency 대기 (ex. config.RL = 20 → 20 cycle 기다린 후 data를 bus에 올림)
  2. Data bus timing 관리 (= BL/2 설정) → Bus에 올라간 data는 BL/2 cycle간 transfer
  3. 실제로 BL/2 cycle만큼 대기 후 완료 시 memoryController에게 완료를 알림

```cpp
void MultiChannelMemorySystem::actual_update()
{
    if (currentClockCycle == 0)
    {
        InitOutputFiles(traceFilename);
        DEBUG("DRAMSim2 Clock Frequency =" << clockDomainCrosser.clock1
                                           << "Hz, CPU Clock Frequency="
                                           << clockDomainCrosser.clock2 << "Hz");
    }

    if (currentClockCycle > 0 && currentClockCycle % configuration->EPOCH_LENGTH == 0)
    {
        (*csvOut) << "ms" << currentClockCycle * configuration->tCK * 1E-6;
        for (size_t i = 0; i < configuration->NUM_CHANS; i++)
        {
            channels[i]->printStats(false);
        }
        csvOut->finalize();
    }

    for (size_t i = 0; i < configuration->NUM_CHANS; i++)
    {
        channels[i]->update();
    }

    currentClockCycle++;
}
```

MultiChannelMemorySystem.cpp

```cpp
// update the memory systems state
void MemorySystem::update()
{
    // PRINT(" ----------------- Memory System Update ------------------");

    // updates the state of each of the objects
    // NOTE - do not change order
    for (size_t i = 0; i < num_ranks_; i++)
    {
        (*ranks)[i]->update();
    }
```

```cpp
void MemoryController::receiveFromBus(BusPacket* bpacket)
{
    if (bpacket->busPacketType != DATA)
    {
        ERROR("== Error - Memory Controller received a non-DATA bus packet from rank");
        bpacket->print();
        exit(0);
    }

    if (DEBUG_BUS)
    {
        PRINTN(" -- MC Receiving From Data Bus : ");
        bpacket->print();
    }

    // add to return read data queue
    returnTransaction.push_back(
        new Transaction(RETURN_DATA, bpacket->physicalAddress, bpacket->data));

    // this delete statement saves a mindboggling amount of memory
    delete (bpacket);
}
```

```cpp
void Rank::update()
{
    // An outgoing packet is one that is currently sending on the bus
    // do the book keeping for the packet's time left on the bus
    if (outgoingDataPacket != NULL)
    {
        dataCyclesLeft--;
        if (dataCyclesLeft == 0)
        {
            // if the packet is done on the bus, call receiveFromBus and free up
            // the bus
            memoryController->receiveFromBus(outgoingDataPacket);
            outgoingDataPacket = NULL;
        }
    }

    // decrement the counter for all packets waiting to be sent back
    for (size_t i = 0; i < readReturnCountdown.size(); i++) readReturnCountdown[i]--;

    if (readReturnCountdown.size() > 0 && readReturnCountdown[0] == 0)
    {
        // RL time has passed since the read was issued; this packet is
        // ready to go out on the bus

        outgoingDataPacket = readReturnPacket[0];
        dataCyclesLeft = config.BL / 2;

        // remove the packet from the ranks
        readReturnPacket.erase(readReturnPacket.begin());
        readReturnCountdown.erase(readReturnCountdown.begin());

        if (DEBUG_BUS)
        {
            PRINTN(" -- R" << getChanId() << " Issuing On Data Bus : ");
            outgoingDataPacket->print();
            PRINT("");
        }
    }
    pimRank->update();
    pimRank->step();
}
```

Rank.cpp

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → MultiChannelMemorySystem::actual_update() → MemorySystem::update() → MemoryController::update()
- Transaction Queue에 미처 올리지 못한 transaction을 올릴 수 있는 지 확인
- 그 후, memoryController::update()를 호출
- 다음 page에서 memoryController::update() 동작을 분석

```cpp
// update the memory systems state
void MemorySystem::update()
{
    // PRINT(" ----------------- Memory System Update ------------------");

    // updates the state of each of the objects
    // NOTE - do not change order
    for (size_t i = 0; i < num_ranks_; i++)
    {
        (*ranks)[i]->update();
    }

    // pendingTransactions will only have stuff in it if MARSS is adding stuff
    if (pendingTransactions.size() > 0 && memoryController->WillAcceptTransaction())
    {
        memoryController->addTransaction(pendingTransactions.front());
        pendingTransactions.pop_front();
    }
    memoryController->update();

    // simply increments the currentClockCycle field for each object
    for (size_t i = 0; i < num_ranks_; i++)
    {
        (*ranks)[i]->step();
    }
    memoryController->step();
    this->step();

    // PRINT("\n"); // two new lines
}
```

MemorySystem.cpp

```cpp
void MemoryController::update()
{
    updateBankState();
    // check for outgoing command packets and handle countdowns
    if (outgoingCmdPacket != NULL)
    {
        cmdCyclesLeft--;
        if (cmdCyclesLeft == 0)  // packet is ready to be received by rank
        {
            (*ranks)[outgoingCmdPacket->rank]->receiveFromBus(outgoingCmdPacket);
            outgoingCmdPacket = NULL;
        }
    }

    // check for outgoing data packets and handle countdowns
    if (outgoingDataPacket != NULL)
    {
        dataCyclesLeft--;
        if (dataCyclesLeft == 0)
        {
            // inform upper levels that a write is done
            if (parentMemorySystem->WriteDataDone != NULL)
            {
                (*parentMemorySystem->WriteDataDone)(parentMemorySystem->systemID,
                                                     outgoingDataPacket->physicalAddress,
                                                     currentClockCycle);
            }
            parentMemorySystem->numOnTheFlyTransactions--;
            (*ranks)[outgoingDataPacket->rank]->receiveFromBus(outgoingDataPacket);
            outgoingDataPacket = NULL;
        }
    }

    // if any outstanding write data needs to be sent
    // and the appropriate amount of time has passed (WL)
    // then send data on bus
    //
    // write data held in fifo vector along with countdowns
    if (writeDataCountdown.size() > 0)
    {
        for (size_t i = 0; i < writeDataCountdown.size(); i++) writeDataCountdown[i]--;

        if (writeDataCountdown[0] == 0)
        {
            // send to bus and print debug stuff
            if (DEBUG_BUS)
            {
                PRINTN(" -- MC Issuing On Data Bus    : ");
                writeDataToSend[0]->print();
            }

            // queue up the packet to be sent
            if (outgoingDataPacket != NULL)
            {
                ERROR("== Error - Data Bus Collision");
                exit(-1);
            }

            outgoingDataPacket = writeDataToSend[0];
            dataCyclesLeft = config.BL / 2;

            totalTransactions++;

            writeDataCountdown.erase(writeDataCountdown.begin());
            writeDataToSend.erase(writeDataToSend.begin());
        }
    }

    // if its time for a refresh issue a refresh
    // else pop from command queue if it's not empty
    updateRefresh();

    // pass a pointer to a poppedBusPacket
    // function returns true if there is something valid in poppedBusPacket
    if (commandQueue.pop(&poppedBusPacket))
    {
        updateCommandQueue(poppedBusPacket);
    }

    updateTransactionQueue();

    // check for outstanding data to return to the CPU
    if (returnTransaction.size() > 0)
    {
        if (DEBUG_BUS)
            PRINTN(" -- MC Issuing to CPU bus : " << *returnTransaction[0]);
        totalTransactions++;

        bool foundMatch = false;
        // find the pending read transaction to calculate latency
        for (size_t i = 0; i < pendingReadTransactions.size(); i++)
        {
            if (pendingReadTransactions[i]->address == returnTransaction[0]->address)
            {
                unsigned chan, rank, bank, row, col;
                config.addrMapping.addressMapping(returnTransaction[0]->address, chan, rank, bank,
                                                  row, col);
                memoryContStats->insertHistogram(
                    currentClockCycle - pendingReadTransactions[i]->timeAdded, rank, bank);
                // FIXME. Is it correct?
                // memcpy(pendingReadTransactions[i]->data,
                // returnTransaction[0]->data, config.BL * (JEDEC_DATA_BUS_BITS / 8));
                returnReadData(pendingReadTransactions[i]);

                delete pendingReadTransactions[i];
                pendingReadTransactions.erase(pendingReadTransactions.begin() + i);
                foundMatch = true;
                break;
            }
        }
        if (!foundMatch)
        {
            ERROR("Can't find a matching transaction for 0x" << hex << returnTransaction[0]->address
                                                             << dec);
            abort();
        }
        delete returnTransaction[0];
        returnTransaction.erase(returnTransaction.begin());
    }

    // decrement refresh counters
    for (size_t i = 0; i < config.NUM_RANKS; i++) refreshCountdown[i]--;
    for (size_t i = 0; i < config.NUM_BANKS; i++) refreshCountdownBank[i]--;

    // print debug
    printDebugOnUpate();

    commandQueue.step();
}
```

MemoryController.cpp

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → MultiChannelMemorySystem::actual_update() → MemorySystem::update() → MemoryController::update()
  1. DRAM Bank들의 state 변화가 완료되었는지 확인
     - 모든 Rank와 Bank를 순회
     - Count down: 이전 cycle에서 PRECHARGE나 REFRESH 명령이 수행되었다면, 해당 명령의 완료 시간(ex. tRP, tRFC)만큼 counter가 설정되어 있음
     - State transition: 매 cycle 1씩 감소하다가 0이 되면, 해당 Bank의 상태를 Idle로 변경하여 다음 명령(ex. ACTIVATE)을 받을 수 있는 상태로 만듦

```cpp
void MemoryController::update()
{
    updateBankState();
    // check for outgoing command packets and handle countdowns
    if (outgoingCmdPacket != NULL)
    {
        cmdCyclesLeft--;
        if (cmdCyclesLeft == 0)  // packet is ready to be received by rank
        {
            (*ranks)[outgoingCmdPacket->rank]->receiveFromBus(outgoingCmdPacket);
            outgoingCmdPacket = NULL;
        }
    }

    // check for outgoing data packets and handle countdowns
    if (outgoingDataPacket != NULL)
    {
        dataCyclesLeft--;
        if (dataCyclesLeft == 0)
        {
            // inform upper levels that a write is done
            if (parentMemorySystem->WriteDataDone != NULL)
            {
                (*parentMemorySystem->WriteDataDone)(parentMemorySystem->systemID,
                                                     outgoingDataPacket->physicalAddress,
                                                     currentClockCycle);
            }
            parentMemorySystem->numOnTheFlyTransactions--;
            (*ranks)[outgoingDataPacket->rank]->receiveFromBus(outgoingDataPacket);
            outgoingDataPacket = NULL;
        }
    }
```

MemoryController.cpp

```cpp
void MemoryController::updateBankState()
{
    for (size_t i = 0; i < config.NUM_RANKS; i++)
    {
        for (size_t j = 0; j < config.NUM_BANKS; j++)
        {
            if (bankStates[i][j].stateChangeCountdown > 0)
            {
                // decrement counters
                bankStates[i][j].stateChangeCountdown--;

                // if counter has reached 0, change state
                if (bankStates[i][j].stateChangeCountdown == 0)
                {
                    switch (bankStates[i][j].lastCommand)
                    {
                        case REF:
                        case RFCSB:
                        case PRECHARGE:
                            bankStates[i][j].currentBankState = Idle;
                            break;
                        default:
                            break;
                    }
                }
            }
        }
    }
}
```

MemoryController.cpp

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → MultiChannelMemorySystem::actual_update() → MemorySystem::update() → MemoryController::update()

  2. Command Bus

  - DRAM의 Command Bus에 실린 packet이 물리적으로 Rank에 도달하는데 걸리는 cycle을 count
  - outgoingCmdPacket: 현재 Command Bus에 있는 command packet → cmdCyclesLeft: bus transfer latency(ex. tCMD cycle)을 추적
  - cmdCyclesLeft==0 시, Rank에 도착한 것이기에 ranks[...]->receiveFromBus()를 호출 → outgoingCmdPacket = NULL로 초기화

```cpp
void MemoryController::update()
{
    updateBankState();
    // check for outgoing command packets and handle countdowns
    if (outgoingCmdPacket != NULL)
    {
        cmdCyclesLeft--;
        if (cmdCyclesLeft == 0)  // packet is ready to be received by rank
        {
            (*ranks)[outgoingCmdPacket->rank]->receiveFromBus(outgoingCmdPacket);
            outgoingCmdPacket = NULL;
        }
    }

    // check for outgoing data packets and handle countdowns
    if (outgoingDataPacket != NULL)
    {
        dataCyclesLeft--;
        if (dataCyclesLeft == 0)
        {
            // inform upper levels that a write is done
            if (parentMemorySystem->WriteDataDone != NULL)
            {
                (*parentMemorySystem->WriteDataDone)(parentMemorySystem->systemID,
                                                     outgoingDataPacket->physicalAddress,
                                                     currentClockCycle);
            }
            parentMemorySystem->numOnTheFlyTransactions--;
            (*ranks)[outgoingDataPacket->rank]->receiveFromBus(outgoingDataPacket);
            outgoingDataPacket = NULL;
        }
    }
```

MemoryController.cpp

```cpp
void Rank::receiveFromBus(BusPacket* packet)
{
    if (DEBUG_BUS)
    {
        PRINTN(" -- R" << getChanId() << " Receiving On Bus    : ");
        packet->print();
    }
    if (VERIFICATION_OUTPUT)
    {
        packet->print(currentClockCycle, false);
    }
    if (!pimRank->isReservedRA(packet->row))
    {
        check(packet);
        updateState(packet);
    }
    sendToBank(packet);
}
```

Rank.cpp

[cf] packet 구성

```
cmdPacket
패킷 구성:
    busPacketType: ACTIVATE
    physicalAddress: 0x12345600
    rank: 0
    bank: 2
    row: 1024
    column: (사용 안 함)
    data: NULL

WRITE 를 위한 DataPacket
패킷 구성:
    busPacketType: DATA
    physicalAddress: 0x12345600
    rank: 0
    bank: 2
    row: 1024
    column: 8
    data: [0xAF, 0xBE, 0xAD, ...] (64 바이트 분량의 실제 값) or NULL
```

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → MemoryController::update() → Rank::receiveFromBus() → Rank::check()
  - 2. Command Bus → receiveFromBus() → check()
    - 명령이 PIM 연산을 위한 Reserved Row Address를 target하는지 확인 → 일반 DRAM 동작이 아닌 특수 mode라면 표준 timing check를 건너뜀
    - 현재 Bank State에서 이 명령이 실행 가능한지 검사 → 다음 page에 계속

Rank.cpp

```
1.  void Rank::receiveFromBus(BusPacket* packet)
2.  {
3.      if (DEBUG_BUS)
4.      {
5.          PRINTN(" -- R" << getChanId() << " Receiving On Bus    : ");
6.          packet->print();
7.      }
8.      if (VERIFICATION_OUTPUT)
9.      {
10.         packet->print(currentClockCycle, false);
11.     }
12.     if (!pimRank->isReservedRA(packet->row))
13.     {
14.         check(packet);
15.         updateState(packet);
16.     }
17.     sendToBank(packet);
18. }
```

PIMRank.h

```
1.      unsigned inline isReservedRA(unsigned row)
2.      {
3.          return (row & (1 << 13));
4.      }
```

Rank.cpp

```
1.  void Rank::check(BusPacket* packet)
2.  {
3.      if (packet->busPacketType == REF)
4.      {
5.          for (size_t i = 0; i < config.NUM_BANKS; i++)
6.          {
7.              if (bankStates[i].currentBankState != Idle)
8.              {
9.                  ERROR("== Error - ch " << getChanId() << " ra" << getRankId()
10.                                          << " received a REF when not allowed");
11.                 exit(-1);
12.             }
13.         }
14.     }
15.     else if (mode_ == dramMode::SB)
16.     {
17.         checkBank(packet->busPacketType, packet->bank, packet->row);
18.     }
19.     else
20.     {
21.         for (int bank = (packet->bank % 2); bank < config.NUM_BANKS; bank += 2)
22.             checkBank(packet->busPacketType, bank, packet->row);
23.     }
24. }
25.
```

Rank.cpp

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → MemoryController::update() → Rank::receiveFromBus() → Rank::check()
  - 현재 Bank State에서 이 명령이 실행 가능한지 검사
    1. REF: 모든 Bank가 Idle 상태(open row가 없는 상태) → RowActive 상태인 Bank가 있다면 REFRESH 허용 X
    2. Single Bank mode(= 일반적인 DRAM 동작) 검사 → packet에 명시된 단 1개의 Bank에 대해서만 timing을 check
    3. HAB/PIM Mode 검사 (= Multi Bank Mode) → 여러 Bank를 동시에 제어 (모든 짝수 Bank 또는 모든 홀수 Bank를 순회)

```cpp
void Rank::check(BusPacket* packet)
{
    if (packet->busPacketType == REF)
    {
        for (size_t i = 0; i < config.NUM_BANKS; i++)
        {
            if (bankStates[i].currentBankState != Idle)
            {
                ERROR("== Error - ch " << getChanId() << " ra" << getRankId()
                                       << " received a REF when not allowed");
                exit(-1);
            }
        }
    }
    else if (mode_ == dramMode::SB)
    {
        checkBank(packet->busPacketType, packet->bank, packet->row);
    }
    else
    {
        for (int bank = (packet->bank % 2); bank < config.NUM_BANKS; bank += 2)
            checkBank(packet->busPacketType, bank, packet->row);
    }
}
```

Rank.cpp

```cpp
void Rank::checkBank(BusPacketType type, int bank, int row)
{
    switch (type)
    {
        case READ:
            if (bankStates[bank].currentBankState != RowActive ||
                currentClockCycle < bankStates[bank].nextRead ||
                row != bankStates[bank].openRowAddress)
            {
                ERROR("== Error - ch " << getChanId() << " ra" << getRankId() << " ba" << bank
                                       << " received a READ when not allowed @ "
                                       << currentClockCycle);
                exit(-1);
            }
            break;
        case WRITE:
            if (bankStates[bank].currentBankState != RowActive ||
                currentClockCycle < bankStates[bank].nextWrite ||
                row != bankStates[bank].openRowAddress)
            {
                ERROR("== Error - ch " << getChanId() << " ra" << getRankId() << " ba" << bank
                                       << " received a WRITE when not allowed @ "
                                       << currentClockCycle);
                bankStates[bank].print();
                exit(-1);
            }
            break;
        case ACTIVATE:
            if (bankStates[bank].currentBankState != Idle ||
                currentClockCycle < bankStates[bank].nextActivate)
            {
                ERROR("== Error - ch " << getChanId() << " ra" << getRankId() << " ba" << bank
                                       << " received a ACT when not allowed @ "
                                       << currentClockCycle);
                bankStates[bank].print();
                exit(-1);
            }
            break;
        case PRECHARGE:
            if (bankStates[bank].currentBankState != RowActive ||
                currentClockCycle < bankStates[bank].nextPrecharge)
            {
                ERROR("== Error - ch " << getChanId() << " ra" << getRankId() << " ba" << bank
                                       << " received a PRE when not allowed @ "
                                       << currentClockCycle);
                exit(-1);
            }
            break;
        case DATA:
            break;
        default:
            ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
            exit(-1);
            break;
    }
}
```

Rank.cpp

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → MemoryController::update() → Rank::receiveFromBus() → Rank::updateState()
  - 2. Command Bus → receiveFromBus() → updateState()
    - updateState() : command을 받을 수 있으므로, 다음 command가 언제 들어올 수 있는지 timing을 update (현재 command는 실행될 예정 → timing update 필요)
      - updateBank()
        1. REFRESH 시, Refresh 중에는 ACTIVATE command를 issue 못하기에, ACTIVATE command를 언제부터 issue가능한지 timing update
        - 다음 page에 계속

Rank.cpp

```
1.  void Rank::receiveFromBus(BusPacket* packet)
2.  {
3.      if (DEBUG_BUS)
4.      {
5.          PRINTN(" -- R" << getChanId() << " Receiving On Bus    : ");
6.          packet->print();
7.      }
8.      if (VERIFICATION_OUTPUT)
9.      {
10.         packet->print(currentClockCycle, false);
11.     }
12.     if (!pimRank->isReservedRA(packet->row))
13.     {
14.         check(packet);
15.         updateState(packet);
16.     }
17.     sendToBank(packet);
18. }
```

Rank.cpp

```
1.  void Rank::updateState(BusPacket* packet)
2.  {
3.      auto addrMapping = config.addrMapping;
4.      if (packet->busPacketType == REF)
5.      {
6.          refreshWaiting = false;
7.          for (size_t i = 0; i < config.NUM_BANKS; i++)
8.          {
9.              bankStates[i].nextActivate = currentClockCycle + config.tRFC;
10.         }
11.     }
12.     else if (mode_ == dramMode::SB)
13.     {
14.         for (int bank = 0; bank < config.NUM_BANKS; bank++)
15.         {
16.             updateBank(packet->busPacketType, bank, packet->row, bank == packet->bank,
17.                        addrMapping.isSameBankgroup(bank, packet->bank));
18.         }
19.     }
20.     else
21.     {
22.         for (int bank = 0; bank < config.NUM_BANKS; bank++)
23.         {
24.             updateBank(packet->busPacketType, bank, packet->row, (bank % 2) == packet->bank, true);
25.         }
26.     }
27. }
```

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → MemoryController::update() → Rank::receiveFromBus() → Rank::updateState() → Rank::updateBank()
  - updateBank()
    2. Single Bank mode(= 일반적인 DRAM 동작) 검사 → 모든 Bank에 대해 updateBank를 호출하되, 내 명령의 직접적인 대상인지 정보를 넘김
       - 대상 Bank: 명령에 따른 직접적인 latency(ex. tRP, tRC) 적용, Bank의 state를 변경 (ex. Idle → RowActive)
       - 같은 Bank Group: Bank Group 간 timing(ex. tRRDL, tCCDL) 적용
       - 다른 Bank Group: 최소한의 간섭 timing(ex. tRRDS, tCCDS) 적용
       - 다음 page에 계속

Rank.cpp

```
1. void Rank::updateState(BusPacket* packet)
2. {
3.     auto addrMapping = config.addrMapping;
4.     if (packet->busPacketType == REF)
5.     {
6.         refreshWaiting = false;
7.         for (size_t i = 0; i < config.NUM_BANKS; i++)
8.         {
9.             bankStates[i].nextActivate = currentClockCycle + config.tRFC;
10.        }
11.    }
12.    else if (mode_ == dramMode::SB)
13.    {
14.        for (int bank = 0; bank < config.NUM_BANKS; bank++)
15.        {
16.            updateBank(packet->busPacketType, bank, packet->row, bank == packet->bank,
17.                       addrMapping.isSameBankgroup(bank, packet->bank));
18.        }
19.    }
20.    else
21.    {
22.        for (int bank = 0; bank < config.NUM_BANKS; bank++)
23.        {
24.            updateBank(packet->busPacketType, bank, packet->row, (bank % 2) == packet->bank, true);
25.        }
26.    }
27. }
```

Rank.cpp

```
1. void Rank::updateBank(BusPacketType type, int bank, int row, bool targetBank, bool targetBankgroup)
2. {
3.     switch (type)
4.     {
5.         case READ:
6.             if (targetBank)
7.                 bankStates[bank].nextPrecharge = max(bankStates[bank].nextPrecharge,
8.                                                      currentClockCycle + config.READ_TO_PRE_DELAY);
9.
10.            if (targetBankgroup)
11.            {
12.                bankStates[bank].nextRead =
13.                    max(bankStates[bank].nextRead,
14.                        currentClockCycle + max(config.tCCDL, config.BL / 2));
15.            }
16.            else
17.            {
18.                bankStates[bank].nextRead =
19.                    max(bankStates[bank].nextRead,
20.                        currentClockCycle + max(config.tCCDS, config.BL / 2));
21.            }
22.            bankStates[bank].nextWrite =
23.                max(bankStates[bank].nextWrite, currentClockCycle + config.READ_TO_WRITE_DELAY);
24.
25.            break;
26.        case WRITE:
27.            // update state table
28.            if (targetBank)
29.                bankStates[bank].nextPrecharge = max(bankStates[bank].nextPrecharge,
30.                                                     currentClockCycle + config.WRITE_TO_PRE_DELAY);
31.            if (targetBankgroup)
32.            {
33.                bankStates[bank].nextRead =
34.                    max(bankStates[bank].nextRead,
35.                        currentClockCycle + config.WRITE_TO_READ_DELAY_B_LONG);
36.                bankStates[bank].nextWrite =
37.                    max(bankStates[bank].nextWrite,
38.                        currentClockCycle + max(config.BL / 2, config.tCCDL));
39.            }
40.            else
41.            {
42.                bankStates[bank].nextRead =
43.                    max(bankStates[bank].nextRead,
44.                        currentClockCycle + config.WRITE_TO_READ_DELAY_B_SHORT);
45.                bankStates[bank].nextWrite =
46.                    max(bankStates[bank].nextWrite,
47.                        currentClockCycle + max(config.BL / 2, config.tCCDS));
48.            }
49.            break;
50.        case ACTIVATE:
51.            if (targetBank)
52.            {
53.                bankStates[bank].currentBankState = RowActive;
54.                bankStates[bank].nextActivate = currentClockCycle + config.tRC;
55.                bankStates[bank].openRowAddress = row;
56.                bankStates[bank].nextWrite = currentClockCycle + (config.tRCDWR - config.AL);
57.                bankStates[bank].nextRead = currentClockCycle + (config.tRCDRD - config.AL);
58.                bankStates[bank].nextPrecharge = currentClockCycle + config.tRAS;
59.            }
60.            else
61.            {
62.                bankStates[bank].nextActivate =
63.                    (targetBankgroup)
64.                        ? max(bankStates[bank].nextActivate, currentClockCycle + config.tRRDL)
65.                        : max(bankStates[bank].nextActivate, currentClockCycle + config.tRRDS);
66.            }
67.            break;
68.
69.        case PRECHARGE:
70.            if (targetBank)
71.            {
72.                bankStates[bank].currentBankState = Idle;
73.                bankStates[bank].nextActivate =
74.                    max(bankStates[bank].nextActivate, currentClockCycle + config.tRP);
75.            }
76.            break;
77.
78.        case DATA:
79.            break;
80.
81.        default:
82.            ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
83.            exit(0);
84.            break;
85.    }
86. }
```

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → MemoryController::update() → Rank::receiveFromBus() → Rank::updateState() → Rank::updateBank()
  - updateBank()

3. HAB/PIM mode (= Multi Bank Mode) → 짝수 또는 홀수 번호의 Bank들을 동시에 Target Bank로 취급하여 업데이트
   - 대상 Bank: 명령에 따른 직접적인 latency(ex. tRP, tRC) 적용, Bank의 state를 변경 (ex. Idle → RowActive)
   - 같은 Bank Group: Bank Group 간 timing(ex. tRRDL, tCCDL) 적용
   - 다른 Bank Group: 최소한의 간섭 timing(ex. tRRDS, tCCDS) 적용

Rank.cpp

```
1. void Rank::updateState(BusPacket* packet)
2. {
3.     auto addrMapping = config.addrMapping;
4.     if (packet->busPacketType == REF)
5.     {
6.         refreshWaiting = false;
7.         for (size_t i = 0; i < config.NUM_BANKS; i++)
8.         {
9.             bankStates[i].nextActivate = currentClockCycle + config.tRFC;
10.        }
11.    }
12.    else if (mode_ == dramMode::SB)
13.    {
14.        for (int bank = 0; bank < config.NUM_BANKS; bank++)
15.        {
16.            updateBank(packet->busPacketType, bank, packet->row, bank == packet->bank,
17.                       addrMapping.isSameBankgroup(bank, packet->bank));
18.        }
19.    }
20.    else
21.    {
22.        for (int bank = 0; bank < config.NUM_BANKS; bank++)
23.        {
24.            updateBank(packet->busPacketType, bank, packet->row, (bank % 2) == packet->bank, true);
25.        }
26.    }
27. }
```

Rank.cpp

```
1. void Rank::updateBank(BusPacketType type, int bank, int row, bool targetBank, bool targetBankgroup)
2. {
3.     switch (type)
4.     {
5.         case READ:
6.             if (targetBank)
7.                 bankStates[bank].nextPrecharge = max(bankStates[bank].nextPrecharge,
8.                                                      currentClockCycle + config.READ_TO_PRE_DELAY);
9.
10.            if (targetBankgroup)
11.            {
12.                bankStates[bank].nextRead =
13.                    max(bankStates[bank].nextRead,
14.                        currentClockCycle + max(config.tCCDL, config.BL / 2));
15.            }
16.            else
17.            {
18.                bankStates[bank].nextRead =
19.                    max(bankStates[bank].nextRead,
20.                        currentClockCycle + max(config.tCCDS, config.BL / 2));
21.            }
22.            bankStates[bank].nextWrite =
23.                max(bankStates[bank].nextWrite, currentClockCycle + config.READ_TO_WRITE_DELAY);
24.
25.            break;
26.        case WRITE:
27.            // update state table
28.            if (targetBank)
29.                bankStates[bank].nextPrecharge = max(bankStates[bank].nextPrecharge,
30.                                                     currentClockCycle + config.WRITE_TO_PRE_DELAY);
31.            if (targetBankgroup)
32.            {
33.                bankStates[bank].nextRead =
34.                    max(bankStates[bank].nextRead,
35.                        currentClockCycle + config.WRITE_TO_READ_DELAY_B_LONG);
36.                bankStates[bank].nextWrite =
37.                    max(bankStates[bank].nextWrite,
38.                        currentClockCycle + max(config.BL / 2, config.tCCDL));
39.            }
40.            else
41.            {
42.                bankStates[bank].nextRead =
43.                    max(bankStates[bank].nextRead,
44.                        currentClockCycle + config.WRITE_TO_READ_DELAY_B_SHORT);
45.                bankStates[bank].nextWrite =
46.                    max(bankStates[bank].nextWrite,
47.                        currentClockCycle + max(config.BL / 2, config.tCCDS));
48.            }
49.            break;
50.        case ACTIVATE:
51.            if (targetBank)
52.            {
53.                bankStates[bank].currentBankState = RowActive;
54.                bankStates[bank].nextActivate = currentClockCycle + config.tRC;
55.                bankStates[bank].openRowAddress = row;
56.                bankStates[bank].nextWrite = currentClockCycle + (config.tRCDWR - config.AL);
57.                bankStates[bank].nextRead = currentClockCycle + (config.tRCDRD - config.AL);
58.                bankStates[bank].nextPrecharge = currentClockCycle + config.tRAS;
59.            }
60.            else
61.            {
62.                bankStates[bank].nextActivate =
63.                    (targetBankgroup)
64.                        ? max(bankStates[bank].nextActivate, currentClockCycle + config.tRRDL)
65.                        : max(bankStates[bank].nextActivate, currentClockCycle + config.tRRDS);
66.            }
67.            break;
68.
69.        case PRECHARGE:
70.            if (targetBank)
71.            {
72.                bankStates[bank].currentBankState = Idle;
73.                bankStates[bank].nextActivate =
74.                    max(bankStates[bank].nextActivate, currentClockCycle + config.tRP);
75.            }
76.            break;
77.
78.        case DATA:
79.            break;
80.
81.        default:
82.            ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
83.            exit(0);
84.            break;
85.    }
86. }
```

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → MemoryController::update() → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → receiveFromBus()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행
    - 다음 page에서 sendToBank() 분석

Rank.cpp

```
1.  void Rank::receiveFromBus(BusPacket* packet)
2.  {
3.      if (DEBUG_BUS)
4.      {
5.          PRINTN(" -- R" << getChanId() << " Receiving On Bus    : ");
6.          packet->print();
7.      }
8.      if (VERIFICATION_OUTPUT)
9.      {
10.         packet->print(currentClockCycle, false);
11.     }
12.     if (!pimRank->isReservedRA(packet->row))
13.     {
14.         check(packet);
15.         updateState(packet);
16.     }
17.     sendToBank(packet);
18. }
```

Rank.cpp

```
1.  void Rank::sendToBank(BusPacket* packet)
2.  {
3.      switch (packet->busPacketType)
4.      {
5.          case READ:
6.              if (mode_ == dramMode::SB)
7.                  readSb(packet);
8.              else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                  pimRank->doPIM(packet);
10.             else
11.                 pimRank->readHab(packet);
12.             packet->busPacketType = DATA;
13.             readReturnPacket.push_back(packet);
14.             readReturnCountdown.push_back(config.RL);
15.             break;
16.         case WRITE:
17.             if (mode_ == dramMode::SB)
18.                 writeSb(packet);
19.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                 pimRank->doPIM(packet);
21.             else
22.                 pimRank->writeHab(packet);
23.             delete (packet);
24.             break;
25.         case ACTIVATE:
26.             if (DEBUG_CMD_TRACE)
27.             {
28.                 PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.             }
30.             if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                 packet->column == 0x1f)
32.             {
33.                 abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                 abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                 abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                 abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                 if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                     (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                 {
41.                     abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                     mode_ = dramMode::HAB;
43.                     if (DEBUG_CMD_TRACE)
44.                     {
45.                         PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                     }
47.                 }
48.             }
49.             delete (packet);
50.             break;
51.         case PRECHARGE:
52.             if (DEBUG_CMD_TRACE)
53.             {
54.                 if (mode_ == dramMode::SB || packet->bank < 2)
55.                 {
56.                     PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                 }
58.                 else
59.                 {
60.                     PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                 }
62.             }
63.
64.             if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
65.             {
66.                 sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
67.                 sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
68.
69.                 if (sbmr1_ && sbmr2_)
70.                 {
71.                     sbmr1_ = sbmr2_ = false;
72.                     mode_ = dramMode::SB;
73.                     if (DEBUG_CMD_TRACE)
74.                     {
75.                         PRINTC(RED, OUTLOG_CH_RA("SB mode"));
76.                     }
77.                 }
78.             }
79.
80.             delete (packet);
81.             break;
82.
83.         case REF:
84.             refreshWaiting = false;
85.             if (DEBUG_CMD_TRACE)
86.             {
87.                 PRINT(OUTLOG_CH_RA("REF"));
88.             }
89.             delete (packet);
90.             break;
91.
92.         case DATA:
93.             delete (packet);
94.             break;
95.
96.         default:
97.             ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
98.             exit(0);
99.             break;
100.    }
101. }
```

Rank.cpp

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → receiveFromBus() → sendToBank() → readSb()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

<Data movement according to operation mode>

Index	Mode	CMD	RA13	Data Movement
①	Normal	WR	L	Data write to cell arrray
②		WR	H	Not available
③		RD	L	Data read to cell arrray
④		RD	H	Not available
⑤	FIM	WR	L	Data movement from PCU blck to cell array
⑥		WR	H	Data write to PCU register
⑦		RD	L	Data movement from cell array to PCU bock
⑧		RD	H	Not available

command type별 처리 진행

1.1. Read → SB mode 시, readSb()(read Standard Bank) 호출

- PIM Register File(GRF_A, GRF_B)
  - row address가 특정 address를 가리킬 때 수행됨
  - PIMBlock의 general register file을 직접 읽음
  - column address가 범위를 벗어나면 일반 memory 읽음
- 일반 DRAM Memory
  - banks[]의.read() 함수 호출로 Memory 접근
- 발견된 이상한 점
  - banks[]의.read() 함수 호출로 Memory 접근 시
  - isReservedRA() == true 임에도 접근하고 있음 → ISSCC에서 허용 X

```
packet->bank = 0 → pimBlocks[0]
packet->bank = 1 → pimBlocks[0] (같은 블록)
packet->bank = 2 → pimBlocks[1]
packet->bank = 3 → pimBlocks[1] (같은 블록)
...
```

PIMBlock의 구성 예시

```
0x08~0x0f (8~15):  GRF_A (General Register File A) - 8개
  → grfA[0] = column 0x08
  → grfA[1] = column 0x09
  → ...
  → grfA[7] = column 0x0f

0x18~0x1f (24~31): GRF_B (General Register File B) - 8개
  → grfB[0] = column 0x18
  → grfB[1] = column 0x19
  → ...
  → grfB[7] = column 0x1f
```

grfA, grfB의 column address

```
PIM_REG_RA = 0x3fff;
PIM_ABMR_RA = 0x27ff;
PIM_SBMR_RA = 0x2fff;
```

Configuration.h

```
BurstType srf;
BurstType grfA[8];
BurstType grfB[8];
```

PIMBlock.h

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.            }
30.            if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                    (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                    mode_ = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
51.        case PRECHARGE:
52.            if (DEBUG_CMD_TRACE)
53.            {
```

Rank.cpp

```
1. void Rank::readSb(BusPacket* packet)
2. {
3.     if (DEBUG_CMD_TRACE)
4.     {
5.         if (packet->row == config.PIM_REG_RA)
6.         {
7.             if (0x08 <= packet->column && packet->column <= 0x0f)
8.             {
9.                 PRINT(OUTLOG_GRF_A("READ_GRF_A"));
10.            }
11.            else if (0x18 <= packet->column && packet->column <= 0x1f)
12.            {
13.                PRINT(OUTLOG_GRF_B("READ_GRF_B"));
14.            }
15.        }
16.        else if (pimRank->isReservedRA(packet->row))
17.        {
18.            PRINTC(GRAY, OUTLOG_ALL("READ"));
19.        }
20.        else
21.        {
22.            PRINT(OUTLOG_ALL("READ"));
23.        }
24.    }
25.
26. #ifndef NO_STORAGE
27.    if (packet->row == config.PIM_REG_RA)
28.    {
29.        if (0x08 <= packet->column && packet->column <= 0x0f)
30.            *(packet->data) = pimRank->pimBlocks[packet->bank / 2].grfA[packet->column - 0x8];
31.        else if (0x18 <= packet->column && packet->column <= 0x1f)
32.            *(packet->data) = pimRank->pimBlocks[packet->bank / 2].grfB[packet->column - 0x18];
33.        else
34.            banks[packet->bank].read(packet);
35.    }
36.    else
37.        banks[packet->bank].read(packet);
38. #endif
39. }
```

Rank.cpp

```
1. void Bank::read(BusPacket* busPacket)
2. {
3.     shared_ptr<DataStruct> rowHeadNode = rowEntries[busPacket->column];
4.     shared_ptr<DataStruct> foundNode = NULL;
5.     if ((foundNode = Bank::searchForRow(busPacket->row, rowHeadNode)) == NULL)
6.     {
7.     }
8.     else  // found it
9.     {
10.        *(busPacket->data) = foundNode->data;
11.    }
12. }
```

Bank.cpp

```
1. shared_ptr<Bank::DataStruct> Bank::searchForRow(unsigned row, shared_ptr<DataStruct> head)
2. {
3.     while (head != NULL)
4.     {
5.         if (head->row == row)
6.         {
7.             // found it
8.             return head;
9.         }
10.        // keep looking
11.        head = head->next;
12.    }
13.    // if we get here, didn't find it
14.    return NULL;
15. }
```

Bank.cpp

Reference: Hardware_Architecture_and_Software_Stack_ ndustrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → receiveFromBus() → sendToBank() → isToggleCond()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

```
1.  void Rank::sendToBank(BusPacket* packet)
2.  {
3.      switch (packet->busPacketType)
4.      {
5.      case READ:
6.          if (mode_ == dramMode::SB)
7.              readSb(packet);
8.          else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.              pimRank->doPIM(packet);
10.         else
11.             pimRank->readHab(packet);
12.         packet->busPacketType = DATA;
13.         readReturnPacket.push_back(packet);
14.         readReturnCountdown.push_back(config.RL);
15.         break;
16.     case WRITE:
17.         if (mode_ == dramMode::SB)
18.             writeSb(packet);
19.         else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.             ...
21.
22.
23.
24.
25.
26.
27.
28.
29.
30.
31.
32.
33.
34.
35.
36.
37.
38.
39.
40.
41.
42.
43.
44.
45.
46.
47.
48.
49.         delete (packet);
50.         break;
51.     case PRECHARGE:
52.         if (DEBUG_CMD_TRACE)
53.         {
54.             if (mode_ == dramMode::SB || packet->bank < 2)
55.             {
56.                 PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.             }
58.             else
59.             {
60.                 PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.             }
62.         }
63.
64.         if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
65.         {
66.             sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
67.             sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
68.
69.             if (sbmr1_ && sbmr2_)
70.             {
71.                 sbmr1_ = sbmr2_ = false;
72.                 mode_ = dramMode::SB;
73.                 if (DEBUG_CMD_TRACE)
74.                 {
75.                     PRINTC(RED, OUTLOG_CH_RA("SB mode"));
76.                 }
77.             }
84.             refreshWaiting = false;
97.             ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
98.             exit(0);
99.             break;
100.    }
101. }
```

```
1. bool PIMRank::isToggleCond(BusPacket* packet)
2. {
3.     if (pimOpMode_ && !crfExit_)
4.     {
5.         if (toggleRa13h_)
6.         {
7.             if (toggleEvenBank_ && ((packet->bank & 1) == 0))
8.                 return true;
9.             else if (toggleOddBank_ && ((packet->bank & 1) == 1))
10.                return true;
11.            return false;
12.        }
13.        else if (!toggleRa13h_ && !isReservedRA(packet->row))
14.        {
15.            if (toggleEvenBank_ && ((packet->bank & 1) == 0))
16.                return true;
17.            else if (toggleOddBank_ && ((packet->bank & 1) == 1))
18.                return true;
19.            return false;
20.        }
21.        return false;
22.    }
23.    else
24.    {
25.        return false;
26.    }
27. }
```

```
1.  unsigned inline isReservedRA(unsigned row)
2.  {
3.      return (row & (1 << 13));
4.  }
```

```
1. void PIMRank::controlPIM(BusPacket* packet)
2. {
3.     // ...
4.     pimOpMode_ = packet->data->u8Data_[0] & 1;              // Bit 0: PIM 모드 활성화
5.     toggleEvenBank_ = !(packet->data->u8Data_[16] & 1);     // Bit 16: 짝수 뱅크
6.     toggleOddBank_ = !(packet->data->u8Data_[16] & 2);      // Bit 17: 홀수 뱅크
7.     toggleRa13h_ = (packet->data->u8Data_[16] & 4);         // Bit 18: Row Bit 13 필터 모드
8. }
```

command type별 처리 진행

1.2. Read → PIM mode 인지 판별

- PIM mode 판별 → isToggleCond() 호출
  - pimOpMode_ : PIM mode 활성화
  - crfExit : EXIT command가 실행되었으면 1
  - toggleRa13h_ : host가 설정하는 제어 신호
    - 0: 일반 memory만 처리, reserved 영역 제외
    - 1: 모든 Bank의 모든 address를 처리
  - isReservedRA(packet->row)
    - packet의 address가 reseved영역 접근하는지
  - toggleEvenBank & toggleOddBank
    - PIM 연산 시 example
      - toggleEvenBank_ = 1 일 때,
      - Bank 0, 2, 4 (짝수)는 PIM 연산 처리
      - Bank 1, 3, 5 (홀수)는 일반 DRAM 처리
      - toggleRa13h_ = 0 일 때, 해당 packet이 isResercedRA() = 1이면, PIM연산 skip

packet->data->u8Data_[]는 mode 설정에 사용

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

  2. Command Bus → receiveFromBus() → sendToBank() → doPIM()

  - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
```

```
1. void PIMRank::doPIM(BusPacket* packet)
2. {
3.     PIMCmd cCmd;
4.     packet->row = masked2accessibleRA(packet->row);
5.     do
6.     {
7.         cCmd.fromInt(crf.data[pimPC_]);
8.         if (DEBUG_CMD_TRACE)
9.         {
10.            PRINTC(CYAN, string((packet->busPacketType == READ) ? "READ ch" : "WRITE ch")
11.                   << getChanId() << " ra" << getRankId() << " bg"
12.                   << config.addrMapping.bankgroupId(packet->bank) << " b" << packet->bank
13.                   << " r" << packet->row << " c" << packet->column << "|| [" << pimPC_
14.                   << "] " << cCmd.toStr() << " @ " << currentClockCycle);
15.        }
16.
17.        if (cCmd.type_ == PIMCmdType::EXIT)
18.        {
19.            crfExit_ = true;
20.            break;
21.        }
22.        else if (cCmd.type_ == PIMCmdType::JUMP)
23.        {
24.            if (lastJumpIdx_ != pimPC_)
25.            {
26.                if (cCmd.loopCounter_ > 0)
27.                {
28.                    lastJumpIdx_ = pimPC_;
29.                    numJumpToBeTaken_ = cCmd.loopCounter_;
30.                }
31.            }
32.            if (numJumpToBeTaken_ > 0)
33.            {
34.                pimPC_ -= cCmd.loopOffset_;
35.                numJumpToBeTaken_--;
36.            }
37.        }
38.        else
39.        {
40.            if (cCmd.type_ == PIMCmdType::FILL || cCmd.isAuto_)
41.            {
42.                if (lastRepeatIdx_ != pimPC_)
43.                {
44.                    lastRepeatIdx_ = pimPC_;
45.                    numRepeatToBeDone_ = 8 - 1;
46.                }
47.
48.                if (numRepeatToBeDone_ > 0)
49.                {
50.                    pimPC_ -= 1;
51.                    numRepeatToBeDone_--;
52.                }
53.                else
54.                    lastRepeatIdx_ = -1;
55.            }
56.            else if (cCmd.type_ == PIMCmdType::NOP)
57.            {
58.                if (lastRepeatIdx_ != pimPC_)
59.                {
60.                    lastRepeatIdx_ = pimPC_;
61.                    numRepeatToBeDone_ = cCmd.loopCounter_;
62.                }
63.
64.                if (numRepeatToBeDone_ > 0)
65.                {
66.                    pimPC_ -= 1;
67.                    numRepeatToBeDone_--;
68.                }
69.                else
70.                    lastRepeatIdx_ = -1;
71.            }
72.
73.            for (int pimblock_id = 0; pimblock_id < config.NUM_PIM_BLOCKS; pimblock_id++)
74.            {
75.                doPIMBlock(packet, cCmd, pimblock_id);
76.
77.                if (DEBUG_PIM_BLOCK && pimblock_id == 0)
78.                {
79.                    PRINT("[BANK_R]" << packet->data->fp16ToStr());
80.                    PRINT("[CMD]" << bitset<32>(cCmd.toInt()) << "(" << cCmd.toStr() << ")");
81.                    PRINT(pimBlocks[pimblock_id].print());
82.                    PRINT("----------");
83.                }
84.            }
85.            pimPC_++;
86.            // EXIT check
87.            PIMCmd next_cmd;
88.            next_cmd.fromInt(crf.data[pimPC_]);
89.            if (next_cmd.type_ == PIMCmdType::EXIT)
90.                crfExit_ = true;
91.        } while (cCmd.type_ == PIMCmdType::JUMP);
92. }
93.
```

PIMRank.cpp

```
1.  unsigned inline masked2accessibleRA(unsigned row)
2.  {
3.      return (row & ((1 << 13) - 1));
4.  }
```

```
1. void PIMCmd::fromInt(uint32_t val)
2. {
3.     type_ = PIMCmdType(fromBit(val, 4, 28));
4.     switch (type_)
5.     {
6.         case PIMCmdType::EXIT:
7.             break;
8.
9.         case PIMCmdType::NOP:
10.            loopCounter_ = fromBit(val, 11, 0);
11.            break;
12.
13.        case PIMCmdType::JUMP:
14.            loopCounter_ = fromBit(val, 17, 11);
15.            loopOffset_ = fromBit(val, 11, 0);
16.            break;
17.
18.        case PIMCmdType::FILL:
19.        case PIMCmdType::MOV:
20.            dst_ = PIMOpdType(fromBit(val, 3, 25));
21.            src0_ = PIMOpdType(fromBit(val, 3, 22));
22.            isRelu_ = fromBit(val, 1, 12);
23.            dstIdx_ = fromBit(val, 4, 8);
24.            src0Idx_ = fromBit(val, 4, 4);
25.            src1Idx_ = fromBit(val, 4, 0);
26.            break;
27.
28.        case PIMCmdType::MAD:
29.            src2_ = PIMOpdType(fromBit(val, 3, 16));
30.        case PIMCmdType::ADD:
31.        case PIMCmdType::MUL:
32.        case PIMCmdType::MAC:
33.            dst_ = PIMOpdType(fromBit(val, 3, 25));
34.            src0_ = PIMOpdType(fromBit(val, 3, 22));
35.            src1_ = PIMOpdType(fromBit(val, 3, 19));
36.            isAuto_ = fromBit(val, 1, 15);
37.            dstIdx_ = fromBit(val, 4, 8);
38.            src0Idx_ = fromBit(val, 4, 4);
39.            src1Idx_ = fromBit(val, 4, 0);
40.            break;
41.
42.        default:
43.            break;
44.    }
45. }
```

```
1.  uint32_t fromBit(uint32_t val, int bit_len, int bit_pos) const
2.  {
3.      return ((val >> bit_pos) & bitmask(bit_len));
4.  }
```

```
1.  union crf_t
2.  {
3.      uint32_t data[32];
4.      BurstType bst[4];
5.      crf_t()
6.      {
7.          memset(data, 0, sizeof(uint32_t) * 32);
8.      }
9.  } crf;
```

CRF 메모리 레이아웃:

crf.data[0] = 32 비트 명령어 #0
crf.data[1] = 32 비트 명령어 #1
crf.data[2] = 32 비트 명령어 #2
...
crf.data[31] = 32 비트 명령어 #31

최대 32 개 명령어 저장 가능

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.1. masked2accessibleRA(packet->row)

- 13번째 bit를 제외하고 LSB 12bit만 가져옴

1.2.2. pimPC_가 가리키는 command read 및 parsing

- EXIT, NOP, JUMP, FILL, MOV, MAD, ADD, MUL, MAC

1.2.3. command type별 PC값 처리

- EXIT : 종료 (crfExit_ = true)
- JUMP : PC를 loopOffset만큼 뒤로, loopCounter까지 반복
- FILL : 8번 반복 실행 (AUTO = 8번 고정됨)
- NOP : loopCounter만큼 stall

```
(Istruction EXAMPLE)
CRF에 저장된 명령어:

CRF[0] = READ_BANK_A (bank 0 → GRF_A[0])          // 첫 번째 입력 읽기
CRF[1] = READ_BANK_B (bank 1 → GRF_A[1])          // 두 번째 입력 읽기
CRF[2] = AUTO_ADD (GRF_A[0], GRF_A[1] → GRF_B) // 덧셈 (8번 반복)
CRF[3] = WRITE_BANK (GRF_B → bank 0)              // 결과 저장
CRF[4] = EXIT                                     // 종료

실행 흐름:
pimPC=0: CRF[0] 실행 → 8번 반복 (8개 블록)
pimPC=1: CRF[1] 실행
pimPC=2: CRF[2] 실행 (FILL/AUTO이므로 8번 반복)
        pimPC--; numRepeat--; (반복 처리)
pimPC=3: CRF[3] 실행
pimPC=4: EXIT 도달 → crfExit_=true
```

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

  2. Command Bus → . . . → doPIM() → doPIMBlock()

  - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. 각 PIM block의 Processing Unit에서 실제 연산 수행

- 다음 page에서 doPIMBlock()을 상세히 분석

```cpp
void Rank::sendToBank(BusPacket* packet)
{
    switch (packet->busPacketType)
    {
        case READ:
            if (mode_ == dramMode::SB)
                readSb(packet);
            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
                pimRank->doPIM(packet);
            else
        case PRECHARGE:
            if (DEBUG_CMD_TRACE)
            {
                if (mode_ == dramMode::SB || packet->bank < 2)
                {
                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
                }
                else
                {
                    PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
```

PIMRank.cpp

```cpp
void PIMRank::doPIM(BusPacket* packet)
{
    PIMCmd cCmd;
    packet->row = masked2accessibleRA(packet->row);
    do
    {
        cCmd.fromInt(crf.data[pimPC_]);
        if (DEBUG_CMD_TRACE)
        {
            PRINTC(CYAN, string((packet->busPacketType == READ) ? "READ ch" : "WRITE ch")
                         << getChanId() << " ra" << getRankId() << " bg"
                         << config.addrMapping.bankgroupId(packet->bank) << " b" << packet->bank
                         << " r" << packet->row << " c" << packet->column << "|| [" << pimPC_
                         << "] " << cCmd.toStr() << " @ " << currentClockCycle);
        }

        if (cCmd.type_ == PIMCmdType::EXIT)
        {
            crfExit_ = true;
            break;
        }
        else if (cCmd.type_ == PIMCmdType::JUMP)
        {
            if (lastJumpIdx_ != pimPC_)
            {
                if (cCmd.loopCounter_ > 0)
                {
                    lastJumpIdx_ = pimPC_;
                    numJumpToBeTaken_ = cCmd.loopCounter_;
                }
            }
            if (numJumpToBeTaken_ > 0)
            {
                pimPC_ -= cCmd.loopOffset_;
                numJumpToBeTaken_--;
            }
        }
        else
        {
            if (cCmd.type_ == PIMCmdType::FILL || cCmd.isAuto_)
            {
                if (lastRepeatIdx_ != pimPC_)
                {
                    lastRepeatIdx_ = pimPC_;
                    numRepeatToBeDone_ = 8 - 1;
                }

                if (numRepeatToBeDone_ > 0)
                {
                    pimPC_ -= 1;
                    numRepeatToBeDone_--;
                }
                else
                    lastRepeatIdx_ = -1;
            }
            else if (cCmd.type_ == PIMCmdType::NOP)
            {
                if (lastRepeatIdx_ != pimPC_)
                {
                    lastRepeatIdx_ = pimPC_;
                    numRepeatToBeDone_ = cCmd.loopCounter_;
                }

                if (numRepeatToBeDone_ > 0)
                {
                    pimPC_ -= 1;
                    numRepeatToBeDone_--;
                }
                else
                    lastRepeatIdx_ = -1;
            }

            for (int pimblock_id = 0; pimblock_id < config.NUM_PIM_BLOCKS; pimblock_id++)
            {
                doPIMBlock(packet, cCmd, pimblock_id);

                if (DEBUG_PIM_BLOCK && pimblock_id == 0)
                {
                    PRINT("[BANK_R]" << packet->data->fp16ToStr());
                    PRINT("[CMD]" << bitset<32>(cCmd.toInt()) << "(" << cCmd.toStr() << ")");
                    PRINT(pimBlocks[pimblock_id].print());
                    PRINT("----------");
                }
            }
        }
        pimPC_++;
        // EXIT check
        PIMCmd next_cmd;
        next_cmd.fromInt(crf.data[pimPC_]);
        if (next_cmd.type_ == PIMCmdType::EXIT)
            crfExit_ = true;
    } while (cCmd.type_ == PIMCmdType::JUMP);
}
```

PIMRank.cpp

```cpp
void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
{
    if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
    {
        BurstType bst;
        bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;

        readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
        if (cCmd.isRelu_)
        {
            for (int i = 0; i < 16; i++)
                bst.u16Data_[i] = (bst.u16Data_[i] & (1 << 15)) ? 0 : bst.u16Data_[i];
        }
        writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
    }
    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
    {
        BurstType dstBst;
        BurstType src0Bst;
        BurstType src1Bst;
        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);

        if (cCmd.type_ == PIMCmdType::ADD)
            // dstBst = src0Bst + src1Bst;
            pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
        else if (cCmd.type_ == PIMCmdType::MUL)
            // dstBst = src0Bst * src1Bst;
            pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);

        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
    }
    else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
    {
        BurstType dstBst;
        BurstType src0Bst;
        BurstType src1Bst;
        bool is_mac = (cCmd.type_ == PIMCmdType::MAC) ? true : false;

        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, is_mac);
        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
        if (is_mac)
        {
            readOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
            // dstBst = src0Bst * src1Bst + dstBst;
            pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
        }
        else
        {
            BurstType src2Bst;
            readOpd(pimblock_id, src2Bst, cCmd.src2_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
            // dstBst = src0Bst * src1Bst + src2Bst;
            pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst, src2Bst);
        }

        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
    }
    else if (cCmd.type_ == PIMCmdType::NOP && packet->busPacketType == WRITE)
    {
        int grf_id = getGrfIdx(packet->column);
        if (packet->bank == 0)
        {
            *(packet->data) = pimBlocks[pimblock_id].grfA[grf_id];
            rank->banks[pimblock_id * 2].write(packet);  // basically read from bank;
        }
        else if (packet->bank == 1)
        {
            *(packet->data) = pimBlocks[pimblock_id].grfB[grf_id];
            rank->banks[pimblock_id * 2 + 1].write(packet);  // basically read from bank.
        }
    }
}
```

PIMRank.cpp

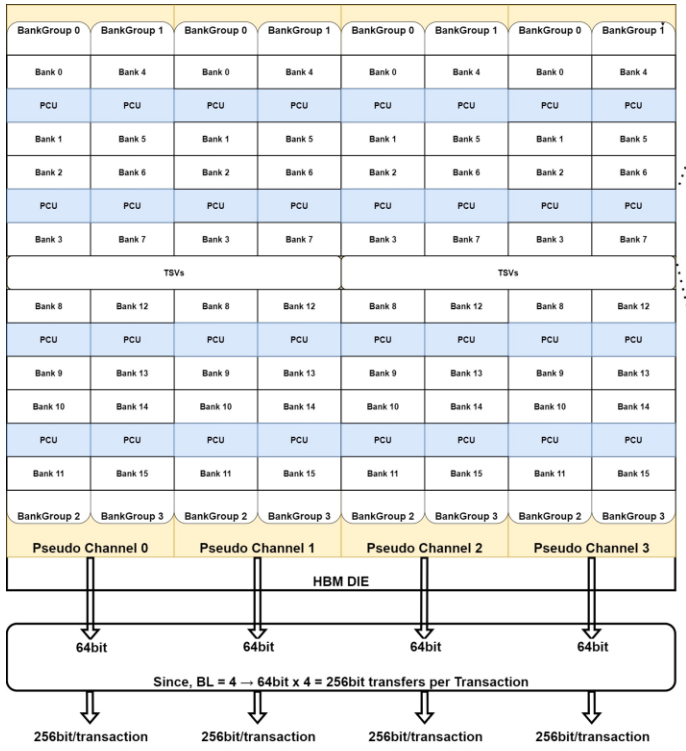
Samsung PIM Simulator hbm read test flow

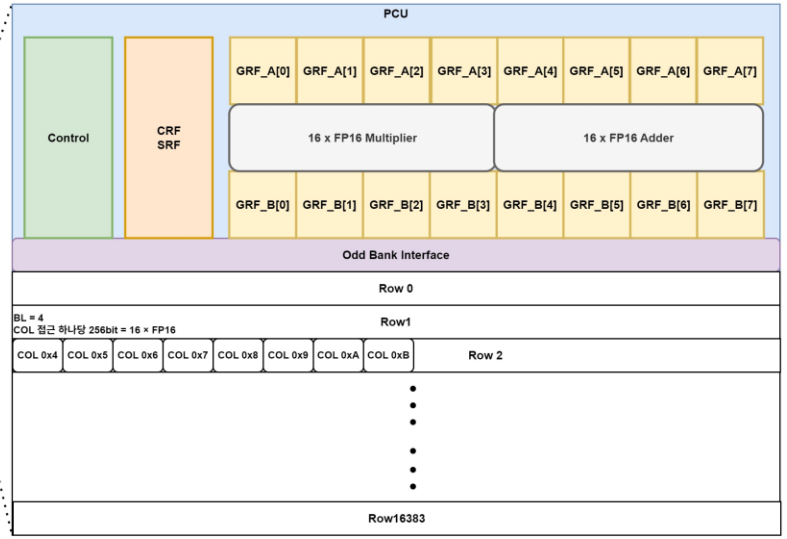
❑ doPIMBlock() 분석 전 HBM2-PIM에 대한 분석

HBM2 spec

```
NUM_BANK_GROUPS=4
NUM_BANKS=16
NUM_COLS=128
NUM_ROWS=16384
NUM_PIM_BLOCKS=8
DEVICE_WIDTH=64
BL=4
RL=20
WL=8
```

- Pseudo channel 당 256bit/transaction
- GRF 1개는 16 x FP16을 저장, GRF_A 8개, GRF_B 8개로 구성
- CRF(command register file), SRF(scalar register file)
- SIMD FP16 Multiplier & Adder





- 아래는 reference의 figure 5.에 제시된 MAC연산 example
- 이를 구체화한 그림
- simulator에서 이러한 operation이 doPIMBlock()에 구현되어 있음
- Operation들이 어떻게 control되는 지 분석 예정

- AAM mode에서 GRF_A/GRF_B의 index가 어떻게 정해지는지

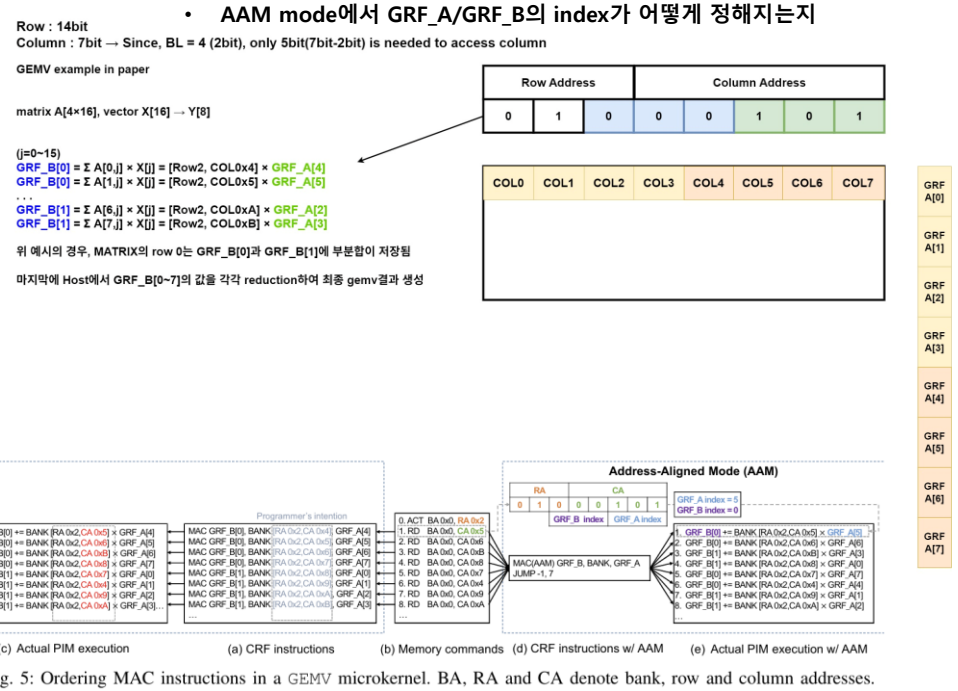


Fig. 5: Ordering MAC instructions in a GEMV microkernel. BA, RA and CA denote bank, row and column addresses.

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → . . . → doPIM() → doPIMBlock()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- readOpd(pb, bst, type, packet, idx, is_auto, is_mac)
  - PIM 명령 시 operand를 다양한 source에서 읽어옴
  - PIMOpdType::A_OUT
    PIMOpdType::M_OUT
    - 이전 계산 결과를 다음 연산의 입력으로 사용
    - Direct Memory Access 없음
  - PIMOpdType::EVEN_BANK
    PIMOpdType::ODD_BANK
    - 실제 DRAM Memory 읽기
    - packet의 row/column 정보 사용하여 접근
  - PIMOpdType::GRF_A → GRF에서 data 읽기
    - is_auto = true: getGrfIdx()로 column addr 중 하위 3bit만 가져옴 (PPT page29 참조)
      (ex. FILL시 같은 주소 8번 반복)
    - is_auto = false: idx를 그대로 사용
      (ex. MOV시 직접 GRF의 idx를 지정하여 사용)

```cpp
void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
{
    if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
    {
        BurstType bst;
        bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;

        readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
        if (cCmd.isRelu_)
        {
            for (int i = 0; i < 16; i++)
                bst.u16Data_[i] = (bst.u16Data_[i] & (1 << 15)) ? 0 : bst.u16Data_[i];
        }
        writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
    }
    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
    {
        BurstType dstBst;
        BurstType src0Bst;
        BurstType src1Bst;
        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);

        if (cCmd.type_ == PIMCmdType::ADD)
            // dstBst = src0Bst + src1Bst;
            pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
        else if (cCmd.type_ == PIMCmdType::MUL)
            // dstBst = src0Bst * src1Bst;
            pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);

        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
    }
```

PIMRank.cpp

```cpp
void PIMRank::readOpd(int pb, BurstType& bst, PIMOpdType type, BusPacket* packet, int idx,
                      bool is_auto, bool is_mac)
{
    idx = getGrfIdx(idx);

    switch (type)
    {
        case PIMOpdType::A_OUT:
            bst = pimBlocks[pb].aOut;
            return;
        case PIMOpdType::M_OUT:
            bst = pimBlocks[pb].mOut;
            return;
        case PIMOpdType::EVEN_BANK:
            if (packet->bank % 2 != 0)
                PRINT("Warning, CRF bank coding and bank id from packet are inconsistent");
            rank->banks[pb * 2].read(packet);  // basically read from bank.
            bst = *(packet->data);
            return;
        case PIMOpdType::ODD_BANK:
            if (packet->bank % 2 == 0)
                PRINT("Warning, CRF bank coding and bank id from packet are inconsistent");
            rank->banks[pb * 2 + 1].read(packet);  // basically read from bank.
            bst = *(packet->data);
            return;
        case PIMOpdType::GRF_A:
            bst = pimBlocks[pb].grfA[(is_auto) ? getGrfIdx(packet->column) : idx];
            return;
        case PIMOpdType::GRF_B:
            if (is_auto)
                bst = pimBlocks[pb].grfB[(is_mac) ? getGrfIdxHigh(packet->row, packet->column)
                                                  : getGrfIdx(packet->column)];
            else
                bst = pimBlocks[pb].grfB[idx];
            return;
        case PIMOpdType::SRF_M:
            bst.set(pimBlocks[pb].srf.fp16Data_[idx]);
            return;
        case PIMOpdType::SRF_A:
            bst.set(pimBlocks[pb].srf.fp16Data_[idx + 8]);
            return;
    }
}
```

Parameter들
- pb : PIMBlock index (0 ~ NUM_PIM_BLOCKS-1)
- bst : 읽어온 data를 저장할 32byte burst
- type : data source 결정
- packet : READ/WRITE/CMD packet
- idx : data index (GRF index, SRF offset 등)
- is_auto : FILL등의 cmd로 자동 address 계산 여부
- is_mac : MAC 명령인지 여부

```cpp
unsigned inline getGrfIdx(unsigned idx)
{
    return idx & 0x7;
}
```

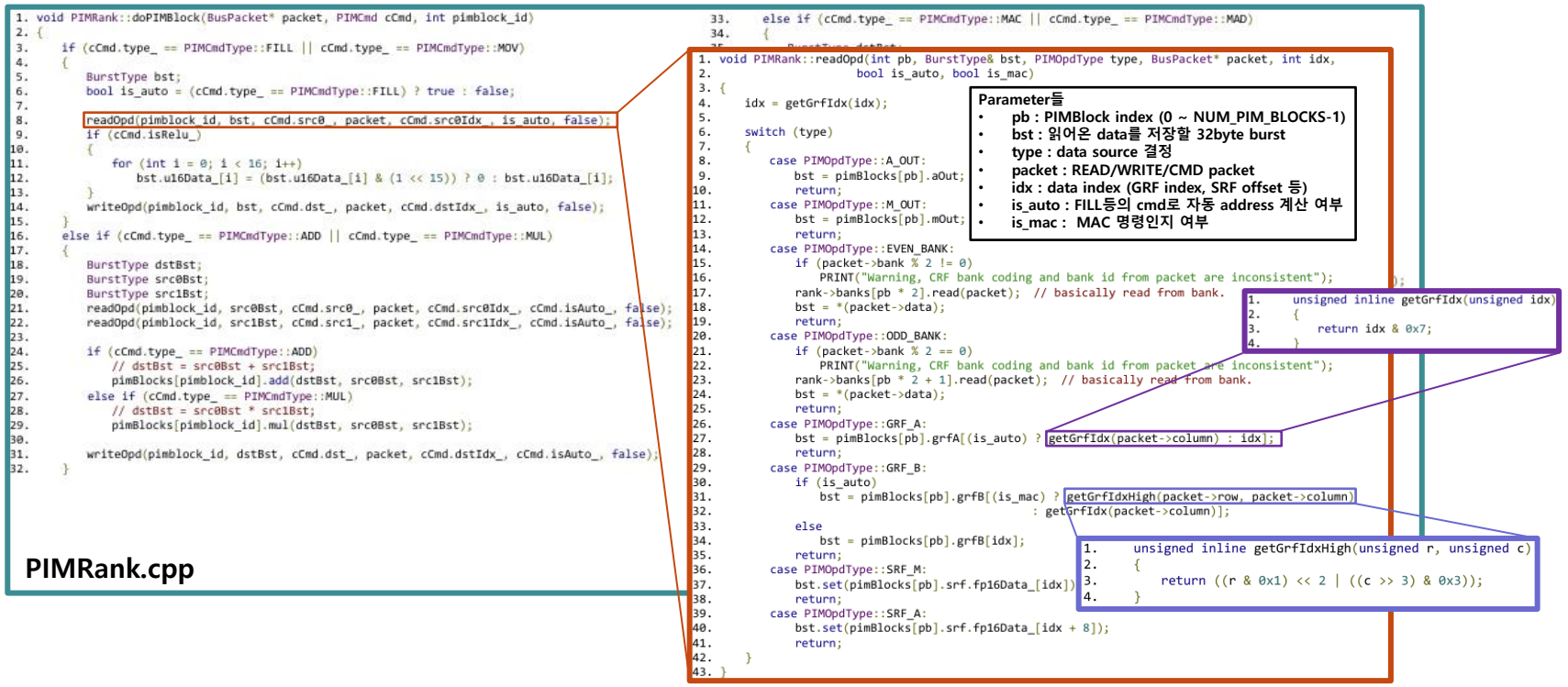
```cpp
unsigned inline getGrfIdxHigh(unsigned r, unsigned c)
{
    return ((r & 0x1) << 2 | ((c >> 3) & 0x3));
}
```

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → . . . → doPIM() → doPIMBlock()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ



PIMRank.cpp

```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
7.
8.         readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
9.         if (cCmd.isRelu_)
10.        {
11.            for (int i = 0; i < 16; i++)
12.                bst.u16Data_[i] = (bst.u16Data_[i] & (1 << 15)) ? 0 : bst.u16Data_[i];
13.        }
14.        writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
15.    }
16.    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
17.    {
18.        BurstType dstBst;
19.        BurstType src0Bst;
20.        BurstType src1Bst;
21.        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
22.        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);
23.
24.        if (cCmd.type_ == PIMCmdType::ADD)
25.            // dstBst = src0Bst + src1Bst;
26.            pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
27.        else if (cCmd.type_ == PIMCmdType::MUL)
28.            // dstBst = src0Bst * src1Bst;
29.            pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);
30.
31.        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
32.    }
```

```
33.    else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
34.    {
```

```
1. void PIMRank::readOpd(int pb, BurstType& bst, PIMOpdType type, BusPacket* packet, int idx,
2.                       bool is_auto, bool is_mac)
3. {
4.     idx = getGrfIdx(idx);
5.
6.     switch (type)
7.     {
8.         case PIMOpdType::A_OUT:
9.             bst = pimBlocks[pb].aOut;
10.            return;
11.        case PIMOpdType::M_OUT:
12.            bst = pimBlocks[pb].mOut;
13.            return;
14.        case PIMOpdType::EVEN_BANK:
15.            if (packet->bank % 2 != 0)
16.                PRINT("Warning, CRF bank coding and bank id from packet are inconsistent");
17.            rank->banks[pb * 2].read(packet);  // basically read from bank.
18.            bst = *(packet->data);
19.            return;
20.        case PIMOpdType::ODD_BANK:
21.            if (packet->bank % 2 == 0)
22.                PRINT("Warning, CRF bank coding and bank id from packet are inconsistent");
23.            rank->banks[pb * 2 + 1].read(packet);  // basically read from bank.
24.            bst = *(packet->data);
25.            return;
26.        case PIMOpdType::GRF_A:
27.            bst = pimBlocks[pb].grfA[(is_auto) ? getGrfIdx(packet->column) : idx];
28.            return;
29.        case PIMOpdType::GRF_B:
30.            if (is_auto)
31.                bst = pimBlocks[pb].grfB[(is_mac) ? getGrfIdxHigh(packet->row, packet->column)
32.                                                  : getGrfIdx(packet->column)];
33.            else
34.                bst = pimBlocks[pb].grfB[idx];
35.            return;
36.        case PIMOpdType::SRF_M:
37.            bst.set(pimBlocks[pb].srf.fp16Data_[idx]);
38.            return;
39.        case PIMOpdType::SRF_A:
40.            bst.set(pimBlocks[pb].srf.fp16Data_[idx + 8]);
41.            return;
42.    }
43. }
```

Parameter들
- pb : PIMBlock index (0 ~ NUM_PIM_BLOCKS-1)
- bst : 읽어온 data를 저장할 32byte burst
- type : data source 결정
- packet : READ/WRITE/CMD packet
- idx : data index (GRF index, SRF offset 등)
- is_auto : FILL등의 cmd로 자동 address 계산 여부
- is_mac : MAC 명령인지 여부

```
1.  unsigned inline getGrfIdx(unsigned idx)
2.  {
3.      return idx & 0x7;
4.  }
```

```
1.  unsigned inline getGrfIdxHigh(unsigned r, unsigned c)
2.  {
3.      return ((r & 0x1) << 2 | ((c >> 3) & 0x3));
4.  }
```

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- readOpd(pb, bst, type, packet, idx, is_auto, is_mac)
  - PIM 명령 시 operand를 다양한 source에서 읽어옴
  - PIMOpdType::GRF_B → GRF에서 data 읽기
    - is_auto = true & is_mac = true
      - getGrfIdxHigh()로 row address 하위 1bit, column address상위 2bit 사용 (PPT page29 참조)
    - is_auto = true & is_mac = false
      - getGrfIdx()로 column addr중 하위 3bit만 가져옴
    - is_auto = false
      - idx를 그대로 사용
  - PIMOpdType::SRF_M
    - srf에서 multiply에 사용할 FP16 data 가져옴
  - PIMOpdType::SRF_M
    - srf에서 add에 사용할 FP16 data 가져옴

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → receiveFromBus()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
7.
8.         readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
9.         if (cCmd.isRelu_)
10.        {
11.            for (int i = 0; i < 16; i++)
12.                bst.u16Data_[i] = (bst.u16Data_[i] & (1 << 15)) ? 0 : bst.u16Data_[i];
13.        }
14.        writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
15.    }
16.    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
17.    {
18.        BurstType dstBst;
19.        BurstType src0Bst;
20.        BurstType src1Bst;
21.        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
22.        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);
23.
24.        if (cCmd.type_ == PIMCmdType::ADD)
25.            // dstBst = src0Bst + src1Bst;
26.            pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
27.        else if (cCmd.type_ == PIMCmdType::MUL)
28.            // dstBst = src0Bst * src1Bst;
29.            pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);
30.
31.        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
32.    }
```

PIMRank.cpp

```
33.    else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
34.    {
```

```
1. void PIMRank::writeOpd(int pb, BurstType& bst, PIMOpdType type, BusPacket* packet, int idx,
2.                        bool is_auto, bool is_mac)
3. {
4.     idx = getGrfIdx(idx);
5.
6.     switch (type)
7.     {
8.         case PIMOpdType::A_OUT:
9.             pimBlocks[pb].aOut = bst;
10.            return;
11.        case PIMOpdType::M_OUT:
12.            pimBlocks[pb].mOut = bst;
13.            return;
14.        case PIMOpdType::EVEN_BANK:
15.            if (packet->bank % 2 != 0)
16.            {
17.                PRINT("CRF bank coding and bank id from packet are inconsistent");
18.            }
19.            *(packet->data) = bst;
20.            rank->banks[pb * 2].write(packet);  // basically read from bank.
21.            return;
22.        case PIMOpdType::ODD_BANK:
23.            if (packet->bank % 2 == 0)
24.            {
25.                PRINT("CRF bank coding and bank id from packet are inconsistent");
26.                exit(-1);
27.            }
28.            *(packet->data) = bst;
29.            rank->banks[pb * 2 + 1].write(packet);  // basically read from bank.
30.            return;
31.        case PIMOpdType::GRF_A:
32.            pimBlocks[pb].grfA[(is_auto) ? getGrfIdx(packet->column) : idx] = bst;
33.            return;
34.        case PIMOpdType::GRF_B:
35.            if (is_auto)
36.                pimBlocks[pb].grfB[(is_mac) ? getGrfIdxHigh(packet->row, packet->column)
37.                                            : getGrfIdx(packet->column)] = bst;
38.            else
39.                pimBlocks[pb].grfB[idx] = bst;
40.            return;
41.        case PIMOpdType::SRF_M:
42.            pimBlocks[pb].srf = bst;
43.            return;
44.        case PIMOpdType::SRF_A:
45.            pimBlocks[pb].srf = bst;
46.            return;
47.    }
48. }
```

Parameter들
- pb : PIMBlock index (0 ~ NUM_PIM_BLOCKS-1)
- bst : write할 32byte burst data
- type : data source 결정
- packet : READ/WRITE/CMD packet
- idx : data dst index (GRF index)
- is_auto : FILL등의 cmd로 자동 address 계산 여부
- is_mac : MAC 명령인지 여부

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

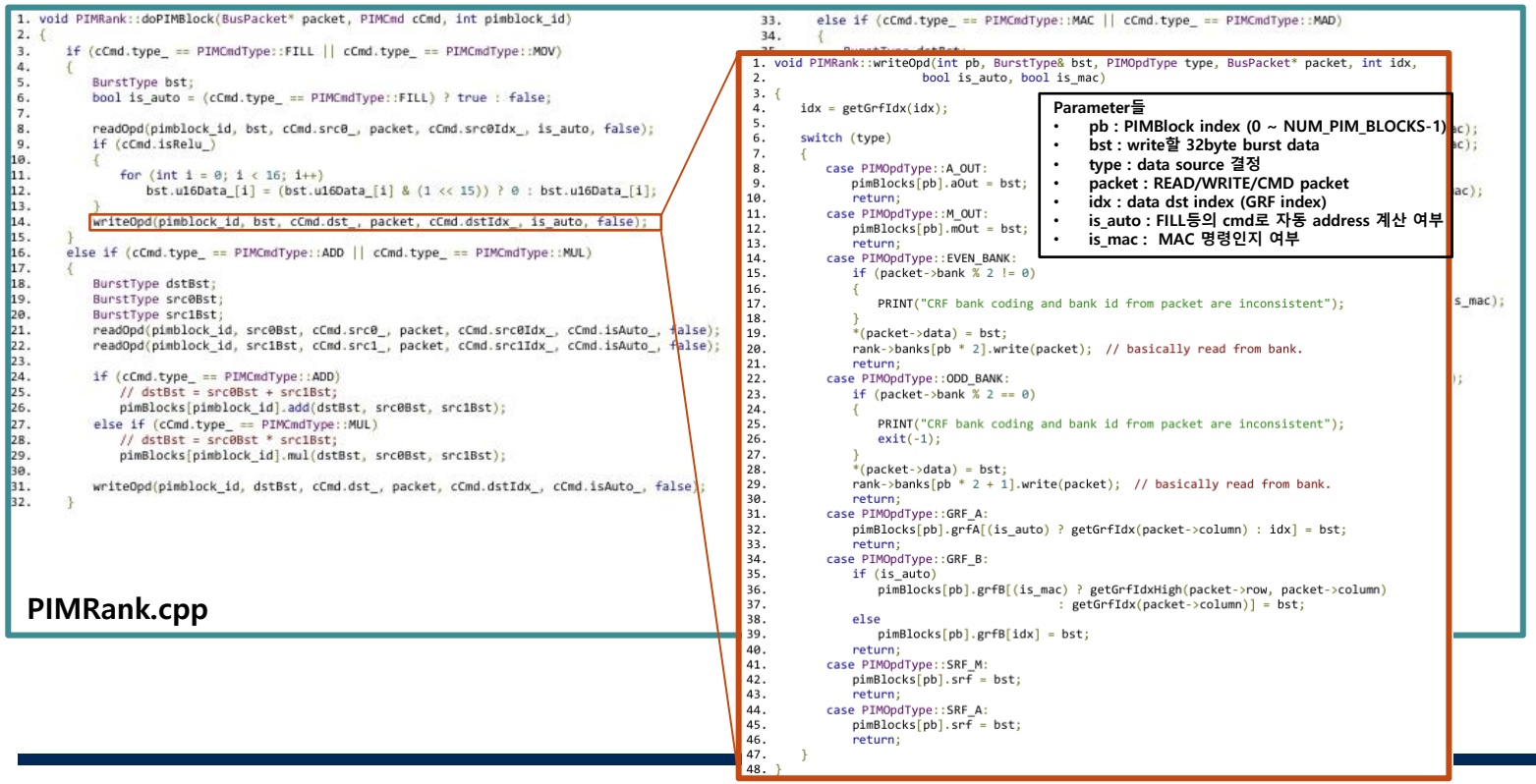
- writeOpd(pb, bst, type, packet, idx, is_auto, is_mac)
  - PIM 연산 후, 계산된 결과값을 dst Operand에 저장
  - PIMOpdType::A_OUT
    PIMOpdType::M_OUT
    - 각 PIM Block 내부에 있는 output buffer에 결과를 저장
  - PIMOpdType::EVEN_BANK
    PIMOpdType::ODD_BANK
    - 실제 DRAM Memory 쓰기
    - packet의 row/column 정보 사용하여 접근
  - PIMOpdType::GRF_A → GRF에 결과 저장
    - is_auto = true: getGrfIdx()로 column addr 중 하위 3bit만 가져옴 (PPT page29 참조) (ex. FILL시 같은 주소 8번 반복)
    - is_auto = false: idx를 그대로 사용 (ex. MOV시 직접 GRF의 idx를 지정하여 사용)

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → receiveFromBus()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ



PIMRank.cpp

```
void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
{
    if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
    {
        BurstType bst;
        bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;

        readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
        if (cCmd.isRelu_)
        {
            for (int i = 0; i < 16; i++)
                bst.u16Data_[i] = (bst.u16Data_[i] & (1 << 15)) ? 0 : bst.u16Data_[i];
        }
        writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
    }
    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
    {
        BurstType dstBst;
        BurstType src0Bst;
        BurstType src1Bst;
        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);

        if (cCmd.type_ == PIMCmdType::ADD)
            // dstBst = src0Bst + src1Bst;
            pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
        else if (cCmd.type_ == PIMCmdType::MUL)
            // dstBst = src0Bst * src1Bst;
            pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);

        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
    }
    else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
    {
```

```
void PIMRank::writeOpd(int pb, BurstType& bst, PIMOpdType type, BusPacket* packet, int idx,
                       bool is_auto, bool is_mac)
{
    idx = getGrfIdx(idx);

    switch (type)
    {
        case PIMOpdType::A_OUT:
            pimBlocks[pb].aOut = bst;
            return;
        case PIMOpdType::M_OUT:
            pimBlocks[pb].mOut = bst;
            return;
        case PIMOpdType::EVEN_BANK:
            if (packet->bank % 2 != 0)
            {
                PRINT("CRF bank coding and bank id from packet are inconsistent");
            }
            *(packet->data) = bst;
            rank->banks[pb * 2].write(packet);  // basically read from bank.
            return;
        case PIMOpdType::ODD_BANK:
            if (packet->bank % 2 == 0)
            {
                PRINT("CRF bank coding and bank id from packet are inconsistent");
                exit(-1);
            }
            *(packet->data) = bst;
            rank->banks[pb * 2 + 1].write(packet);  // basically read from bank.
            return;
        case PIMOpdType::GRF_A:
            pimBlocks[pb].grfA[(is_auto) ? getGrfIdx(packet->column) : idx] = bst;
            return;
        case PIMOpdType::GRF_B:
            if (is_auto)
                pimBlocks[pb].grfB[(is_mac) ? getGrfIdxHigh(packet->row, packet->column)
                                            : getGrfIdx(packet->column)] = bst;
            else
                pimBlocks[pb].grfB[idx] = bst;
            return;
        case PIMOpdType::SRF_M:
            pimBlocks[pb].srf = bst;
            return;
        case PIMOpdType::SRF_A:
            pimBlocks[pb].srf = bst;
            return;
    }
}
```

Parameter들
- pb : PIMBlock index (0 ~ NUM_PIM_BLOCKS-1)
- bst : write할 32byte burst data
- type : data source 결정
- packet : READ/WRITE/CMD packet
- idx : data dst index (GRF index)
- is_auto : FILL등의 cmd로 자동 address 계산 여부
- is_mac : MAC 명령인지 여부

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- writeOpd(pb, bst, type, packet, idx, is_auto, is_mac)
  - PIM 연산 후, 계산된 결과값을 dst Operand에 저장
  - PIMOpdType::GRF_B → GRF에 결과 저장
    - is_auto = true & is_mac = true
      - getGrfIdxHigh()로 row address 하위 1bit, column address상위 2bit 사용 (PPT page29 참조)
    - is_auto = true & is_mac = false
      - getGrfIdx()로 column addr중 하위 3bit만 가져옴
    - is_auto = false
      - idx를 그대로 사용
  - PIMOpdType::SRF_M
  - PIMOpdType::SRF_M
    - Scalar Register File에 값을 update

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → receiveFromBus()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- add(dstBst, src0Bst, src1Bst)
  - 개별 PIM Block이 수행하는 Vector 덧셈 연산 수행
  - Type별 덧셈 수행
    - FP16
      - 16개의 element를 각각 더함
    - FP32
      - 8개의 element를 각각 더함
    - 그 외
      - BurstType operator + 호출

```cpp
void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
{
    if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
    {
        BurstType bst;
        bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;

        readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
        if (cCmd.isRelu_)
        {
            for (int i = 0; i < 16; i++)
                bst.u16Data_[i] = (bst.u16Data_[i] & (1 << 15)) ? 0 : bst.u16Data_[i];
        }
        writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
    }
    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
    {
        BurstType dstBst;
        BurstType src0Bst;
        BurstType src1Bst;
        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);

        if (cCmd.type_ == PIMCmdType::ADD)
            // dstBst = src0Bst + src1Bst;
            pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
        else if (cCmd.type_ == PIMCmdType::MUL)
            // dstBst = src0Bst * src1Bst;
            pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);

        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.i
    }
    else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
    {
        BurstType dstBst;
        BurstType src0Bst;
        BurstType src1Bst;
        bool is_mac = (cCmd.type_ == PIMCmdType::MAC) ? true : false;

        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, is_mac);
        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
        if (is_mac)
        {
            readOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
            // dstBst = src0Bst * src1Bst + dstBst;
            pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
        }
        else
        {
            BurstType src2Bst;
            readOpd(pimblock_id, src2Bst, cCmd.src2_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
            // dstBst = src0Bst * src1Bst + src2Bst;
            pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst, src2Bst);
        }

        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
    }
```

PIMRank.cpp

```cpp
void PIMBlock::add(BurstType& dstBst, BurstType& src0Bst, BurstType& src1Bst)
{
    if (pimPrecision_ == FP16)
    {
        for (int i = 0; i < 16; i++)
        {
            dstBst.fp16Data_[i] = src0Bst.fp16Data_[i] + src1Bst.fp16Data_[i];
        }
    }
    else if (pimPrecision_ == FP32)
    {
        for (int i = 0; i < 8; i++)
        {
            dstBst.fp32Data_[i] = src0Bst.fp32Data_[i] + src1Bst.fp32Data_[i];
        }
    }
    else
        dstBst = src0Bst + src1Bst;
}
```

```cpp
BurstType operator+(const BurstType& rhs) const
{
    BurstType ret;
    for (int i = 0; i < 16; i++)
    {
        ret.fp16Data_[i] = fp16Data_[i] + rhs.fp16Data_[i];
    }
    return ret;
}
```

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → receiveFromBus()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- mul(dstBst, src0Bst, src1Bst)
  - 개별 PIM Block이 수행하는 vector 곱셈 연산 수행
  - Type별 곱셈 수행
    - FP16
      - 16개의 element를 각각 곱함
    - FP32
      - 8개의 element를 각각 곱함
    - 그 외
      - BurstType operator * 호출

```cpp
void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
{
    if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
    {
        BurstType bst;
        bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;

        readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
        if (cCmd.isRelu_)
        {
            for (int i = 0; i < 16; i++)
                bst.u16Data_[i] = (bst.u16Data_[i] & (1 << 15)) ? 0 : bst.u16Data_[i];
        }
        writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
    }
    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
    {
        BurstType dstBst;
        BurstType src0Bst;
        BurstType src1Bst;
        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);

        if (cCmd.type_ == PIMCmdType::ADD)
            // dstBst = src0Bst + src1Bst;
            pimBlocks[pimblock_id].add(dstBst, src0Bst, src1Bst);
        else if (cCmd.type_ == PIMCmdType::MUL)
            // dstBst = src0Bst * src1Bst;
            pimBlocks[pimblock_id].mul(dstBst, src0Bst, src1Bst);

        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, false);
    }
    else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
    {
        BurstType dstBst;
        BurstType src0Bst;
        BurstType src1Bst;
        bool is_mac = (cCmd.type_ == PIMCmdType::MAC) ? true : false;

        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, is_mac);
        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
        if (is_mac)
        {
            readOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
            // dstBst = src0Bst * src1Bst + dstBst;
            pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
        }
        else
        {
            BurstType src2Bst;
            readOpd(pimblock_id, src2Bst, cCmd.src2_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
            // dstBst = src0Bst * src1Bst + src2Bst;
            pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst, src2Bst);
        }

        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
    }
}
```

PIMRank.cpp

```cpp
void PIMBlock::mul(BurstType& dstBst, BurstType& src0Bst, BurstType& src1Bst)
{
    if (pimPrecision_ == FP16)
    {
        for (int i = 0; i < 16; i++)
        {
            dstBst.fp16Data_[i] = src0Bst.fp16Data_[i] * src1Bst.fp16Data_[i];
        }
    }
    else if (pimPrecision_ == FP32)
    {
        for (int i = 0; i < 8; i++)
        {
            dstBst.fp32Data_[i] = src0Bst.fp32Data_[i] * src1Bst.fp32Data_[i];
        }
    }
    else
        dstBst = src0Bst * src1Bst;
}
```

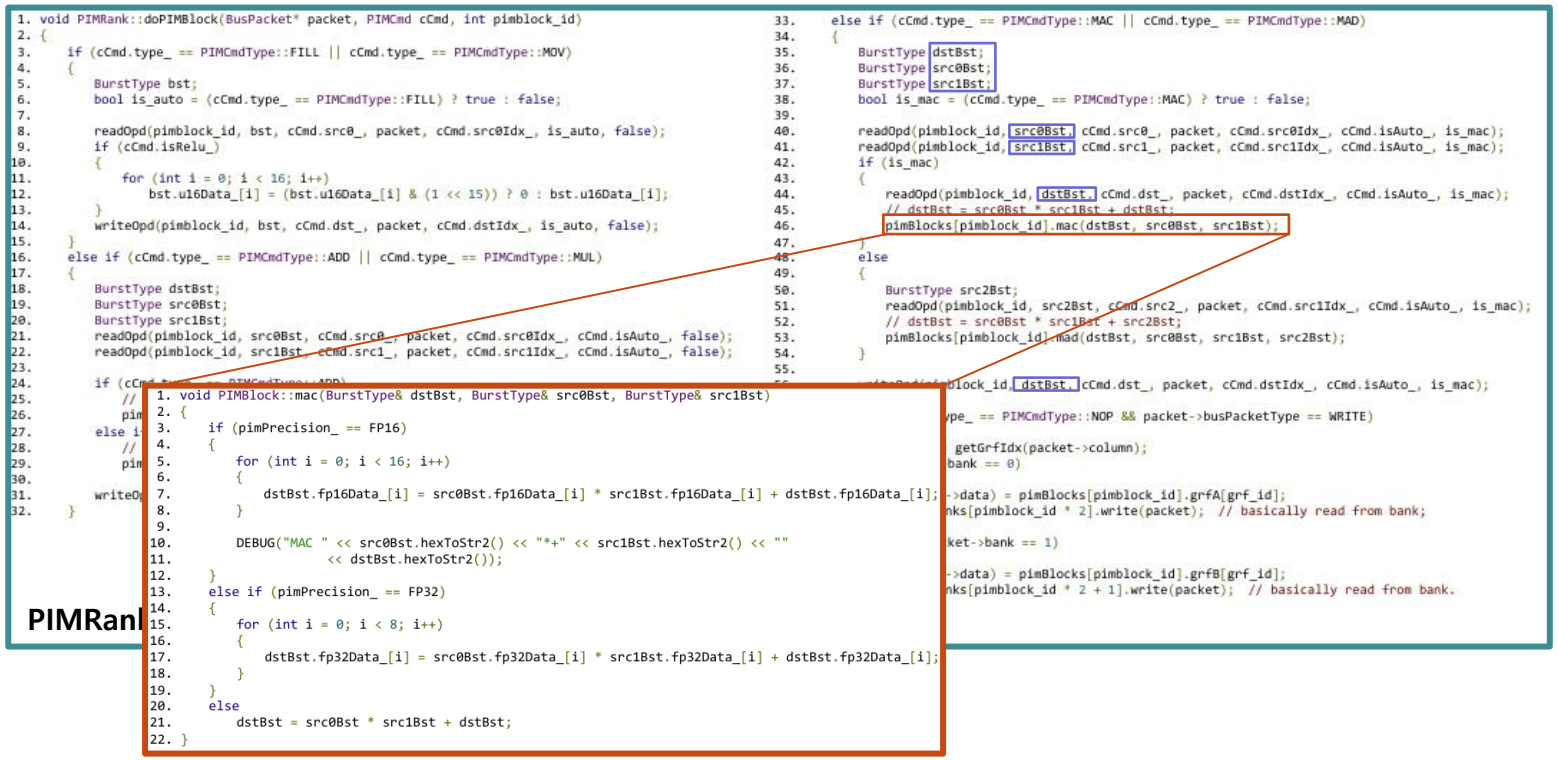
```cpp
BurstType operator*(const BurstType& rhs) const
{
    BurstType ret;
    for (int i = 0; i < 16; i++)
    {
        ret.fp16Data_[i] = fp16Data_[i] * rhs.fp16Data_[i];
    }
    return ret;
}
```

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → receiveFromBus()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ



```
void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
{
    if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
    {
        BurstType bst;
        bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;

        readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
        if (cCmd.isRelu_)
        {
            for (int i = 0; i < 16; i++)
                bst.u16Data_[i] = (bst.u16Data_[i] & (1 << 15)) ? 0 : bst.u16Data_[i];
        }
        writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
    }
    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
    {
        BurstType dstBst;
        BurstType src0Bst;
        BurstType src1Bst;
        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);

        if (cCmd.type_ == PIMCmdType::ADD)
        {
            //
            pim
        else i
            //
            pim

        writeO
    }
```

```
void PIMBlock::mac(BurstType& dstBst, BurstType& src0Bst, BurstType& src1Bst)
{
    if (pimPrecision_ == FP16)
    {
        for (int i = 0; i < 16; i++)
        {
            dstBst.fp16Data_[i] = src0Bst.fp16Data_[i] * src1Bst.fp16Data_[i] + dstBst.fp16Data_[i];
        }

        DEBUG("MAC " << src0Bst.hexToStr2() << "*+" << src1Bst.hexToStr2() << ""
                     << dstBst.hexToStr2());
    }
    else if (pimPrecision_ == FP32)
    {
        for (int i = 0; i < 8; i++)
        {
            dstBst.fp32Data_[i] = src0Bst.fp32Data_[i] * src1Bst.fp32Data_[i] + dstBst.fp32Data_[i];
        }
    }
    else
        dstBst = src0Bst * src1Bst + dstBst;
}
```

```
    else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
    {
        BurstType dstBst;
        BurstType src0Bst;
        BurstType src1Bst;
        bool is_mac = (cCmd.type_ == PIMCmdType::MAC) ? true : false;

        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, is_mac);
        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
        if (is_mac)
        {
            readOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
            // dstBst = src0Bst * src1Bst + dstBst;
            pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
        }
        else
        {
            BurstType src2Bst;
            readOpd(pimblock_id, src2Bst, cCmd.src2_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
            // dstBst = src0Bst * src1Bst + src2Bst;
            pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst, src2Bst);
        }

        block_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);

        ype_ == PIMCmdType::NOP && packet->busPacketType == WRITE)

        getGrfIdx(packet->column);
        bank == 0)

        ->data) = pimBlocks[pimblock_id].grfA[grf_id];
        nks[pimblock_id * 2].write(packet);  // basically read from bank;

        ket->bank == 1)

        ->data) = pimBlocks[pimblock_id].grfB[grf_id];
        nks[pimblock_id * 2 + 1].write(packet);  // basically read from bank.
```

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

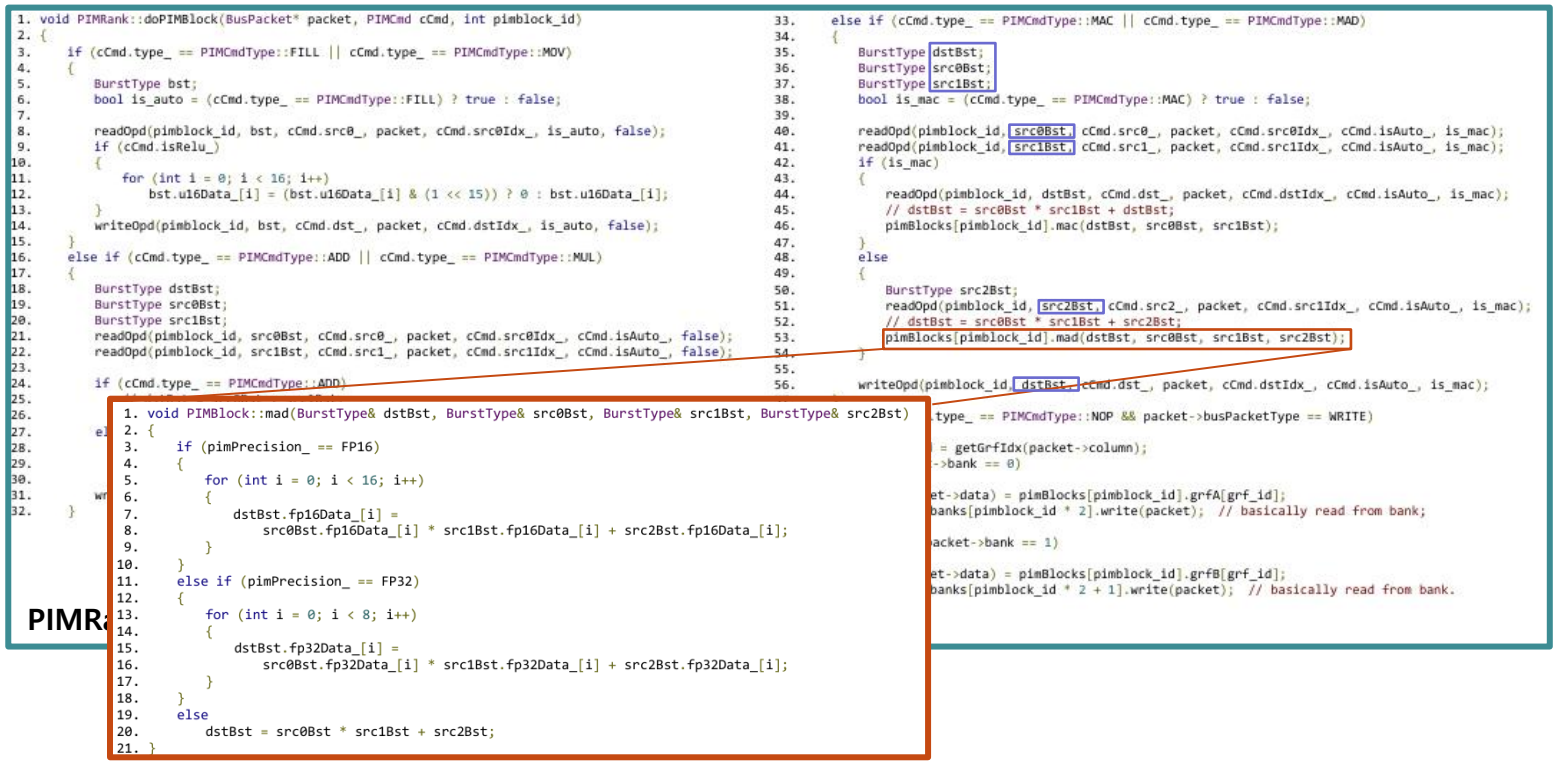
- mac(dstBst, src0Bst, src1Bst)
  - 개별 PIM Block이 수행하는 vector MAC 연산 수행
  - Type별 곱셈 수행
    - FP16
      - 16개의 element를 각각 MAC
    - FP32
      - 8개의 element를 각각 MAC
    - 그 외
      - BurstType operator * 와 + 호출
- mad → c = a * b + c

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → receiveFromBus()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ



```
1. void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
2. {
3.     if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
4.     {
5.         BurstType bst;
6.         bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
7.
8.         readOpd(pimblock_id, bst, cCmd.src0_, packet, cCmd.src0Idx_, is_auto, false);
9.         if (cCmd.isRelu_)
10.        {
11.            for (int i = 0; i < 16; i++)
12.                bst.u16Data_[i] = (bst.u16Data_[i] & (1 << 15)) ? 0 : bst.u16Data_[i];
13.        }
14.        writeOpd(pimblock_id, bst, cCmd.dst_, packet, cCmd.dstIdx_, is_auto, false);
15.    }
16.    else if (cCmd.type_ == PIMCmdType::ADD || cCmd.type_ == PIMCmdType::MUL)
17.    {
18.        BurstType dstBst;
19.        BurstType src0Bst;
20.        BurstType src1Bst;
21.        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, false);
22.        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, false);
23.
24.        if (cCmd.type_ == PIMCmdType::ADD)
25.
26.
27.        el
28.
29.
30.
31.        wr
32.    }
33.    else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
34.    {
35.        BurstType dstBst;
36.        BurstType src0Bst;
37.        BurstType src1Bst;
38.        bool is_mac = (cCmd.type_ == PIMCmdType::MAC) ? true : false;
39.
40.        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, is_mac);
41.        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
42.        if (is_mac)
43.        {
44.            readOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
45.            // dstBst = src0Bst * src1Bst + dstBst;
46.            pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
47.        }
48.        else
49.        {
50.            BurstType src2Bst;
51.            readOpd(pimblock_id, src2Bst, cCmd.src2_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
52.            // dstBst = src0Bst * src1Bst + src2Bst;
53.            pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst, src2Bst);
54.        }
55.
56.        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
```

```
1. void PIMBlock::mad(BurstType& dstBst, BurstType& src0Bst, BurstType& src1Bst, BurstType& src2Bst)
2. {
3.     if (pimPrecision_ == FP16)
4.     {
5.         for (int i = 0; i < 16; i++)
6.         {
7.             dstBst.fp16Data_[i] =
8.                 src0Bst.fp16Data_[i] * src1Bst.fp16Data_[i] + src2Bst.fp16Data_[i];
9.         }
10.    }
11.    else if (pimPrecision_ == FP32)
12.    {
13.        for (int i = 0; i < 8; i++)
14.        {
15.            dstBst.fp32Data_[i] =
16.                src0Bst.fp32Data_[i] * src1Bst.fp32Data_[i] + src2Bst.fp32Data_[i];
17.        }
18.    }
19.    else
20.        dstBst = src0Bst * src1Bst + src2Bst;
21. }
```

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- mad(dstBst, src0Bst, src1Bst , src2Bst)
  - 개별 PIM Block이 수행하는 vector MAD 연산 수행
  - Type별 곱셈 수행
    - FP16
      - 16개의 element를 각각 MAD
    - FP32
      - 8개의 element를 각각 MAD
    - 그 외
      - BurstType operator * 와 + 호출
- mad → d = a * b + c

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()
  - 2. Command Bus → receiveFromBus()
    - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

```
void PIMRank::doPIMBlock(BusPacket* packet, PIMCmd cCmd, int pimblock_id)
{
    if (cCmd.type_ == PIMCmdType::FILL || cCmd.type_ == PIMCmdType::MOV)
    {
        BurstType bst;
        bool is_auto = (cCmd.type_ == PIMCmdType::FILL) ? true : false;
```

```
void Bank::write(const BusPacket* busPacket)
{
    // TODO: move all the error checking to BusPacket so once we have a bus
    // packet,
    //           we know the fields are all legal

    if (busPacket->column >= numCols)
    {
        ERROR("== Error - Bus Packet column " << busPacket->column << " out of bounds");
        exit(-1);
    }

    // head of the list we need to search
    shared_ptr<DataStruct> rowHeadNode = rowEntries[busPacket->column];
    shared_ptr<DataStruct> foundNode = NULL;
    if ((foundNode = Bank::searchForRow(busPacket->row, rowHeadNode)) == NULL)
    {
        // not found
        shared_ptr<DataStruct> newRowNode = make_shared<DataStruct>();
        // DataStruct* newRowNode = (DataStruct*)malloc(sizeof(DataStruct));

        // insert at the head for speed
        // TODO: Optimize this data structure for speedier lookups?
        newRowNode->row = busPacket->row;
        if (busPacket->data)
            newRowNode->data = *(busPacket->data);
        newRowNode->next = rowHeadNode;
        rowEntries[busPacket->column] = newRowNode;
    }
    else
    {
        // found it, just plaster in the new data
        foundNode->data = *(busPacket->data);
        if (DEBUG_BANKS)
        {
            PRINTN(" -- Bank " << busPacket->bank << " writing to physical address 0x" << hex
                               << busPacket->physicalAddress << dec << ":");
            busPacket->printData();
            PRINT("");
        }
    }
}
```

```
    else if (cCmd.type_ == PIMCmdType::MAC || cCmd.type_ == PIMCmdType::MAD)
    {
        BurstType dstBst;
        BurstType src0Bst;
        BurstType src1Bst;
        bool is_mac = (cCmd.type_ == PIMCmdType::MAC) ? true : false;

        readOpd(pimblock_id, src0Bst, cCmd.src0_, packet, cCmd.src0Idx_, cCmd.isAuto_, is_mac);
        readOpd(pimblock_id, src1Bst, cCmd.src1_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
        if (is_mac)
        {
            readOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
            // dstBst = src0Bst * src1Bst + dstBst;
            pimBlocks[pimblock_id].mac(dstBst, src0Bst, src1Bst);
        }
        else
        {
            BurstType src2Bst;
            readOpd(pimblock_id, src2Bst, cCmd.src2_, packet, cCmd.src1Idx_, cCmd.isAuto_, is_mac);
            // dstBst = src0Bst * src1Bst + src2Bst;
            pimBlocks[pimblock_id].mad(dstBst, src0Bst, src1Bst, src2Bst);
        }

        writeOpd(pimblock_id, dstBst, cCmd.dst_, packet, cCmd.dstIdx_, cCmd.isAuto_, is_mac);
    }
    else if (cCmd.type_ == PIMCmdType::NOP && packet->busPacketType == WRITE)
    {
        int grf_id = getGrfIdx(packet->column);
        if (packet->bank == 0)
        {
            *(packet->data) = pimBlocks[pimblock_id].grfA[grf_id];
            rank->banks[pimblock_id * 2].write(packet); // basically read from bank;
        }
        else if (packet->bank == 1)
        {
            *(packet->data) = pimBlocks[pimblock_id].grfB[grf_id];
            rank->banks[pimblock_id * 2 + 1].write(packet);  // basically read from bank.
        }
    }
}
```

command type별 처리 진행

1.2. Read → PIM mode 인지 판별 → doPIM() 수행

1.2.4. doPIMBlock()

- cCmd.type_ == PIMCmdType::NOP && packet->busPacketType == WRITE
- PIM Mode에서 Packet이 Write 동작을 요청했으며, 내부에서 연산을 수행하지 않을 때(=NOP), Register Data를 Memory로 Write Back
- Bank == 0 (= even Bank)인 경우
  - pimBlocks[pimblock_id].grfA[grf_id]에서 data를 꺼내 packet->data에 copy
  - packet->data를 bank에 write
- Bank == 1 (= odd Bank)인 경우
  - pimBlocks[pimblock_id].grfB[grf_id]에서 data를 꺼내 packet->data에 copy
  - packet->data를 bank에 write

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

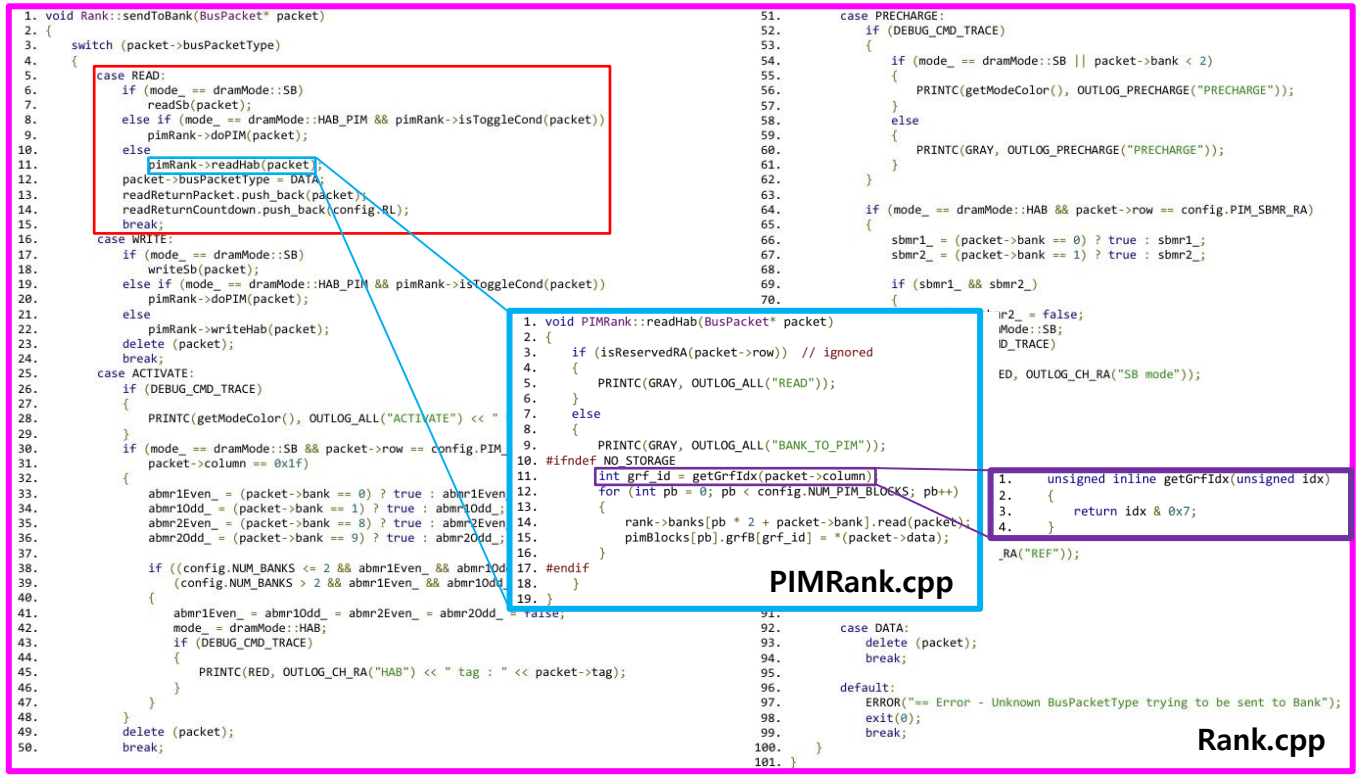
Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

  2. Command Bus → receiveFromBus()

     - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ



command type별 처리 진행

1.3. Read → PIM mode 인지 판별 → 아닐 시 → readHab() 수행

readHab()

- READ 명령 시, PIM 연산을 위해 Bank data를 Reg로 Load
- isReservedRA() == true이면 skip
- isReservedRA() == false 시
  - Read한 Data를 PIM Reg(GRF_B)에 넣음
  - 모든 PIM Blocks에 대해 수행 → Even/Odd Banks 중 선택
  - Even/Odd Banks 는 packet->bank를 통해 선택
    (ex. Packet->bank == 1이면 Odd Banks)

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

  2. Command Bus → receiveFromBus()

     - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case READ

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.            }
30.            if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                    (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                    mode_ = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
51.        case PRECHARGE:
52.            if (DEBUG_CMD_TRACE)
53.            {
54.                if (mode_ == dramMode::SB || packet->bank < 2)
55.                {
56.                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                }
58.                else
59.                {
60.                    PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                }
62.            }
63.
64.            if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
65.            {
66.                sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
67.                sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
68.
69.                if (sbmr1_ && sbmr2_)
70.                {
71.                    sbmr1_ = sbmr2_ = false;
72.                    mode_ = dramMode::SB;
73.                    if (DEBUG_CMD_TRACE)
74.                    {
75.                        PRINTC(RED, OUTLOG_CH_RA("SB mode"));
76.                    }
77.                }
78.            }
79.
80.            delete (packet);
81.            break;
82.
83.        case REF:
84.            refreshWaiting = false;
85.            if (DEBUG_CMD_TRACE)
86.            {
87.                PRINT(OUTLOG_CH_RA("REF"));
88.            }
89.            delete (packet);
90.            break;
91.
92.        case DATA:
93.            delete (packet);
94.            break;
95.
96.        default:
97.            ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
98.            exit(0);
99.            break;
100.    }
101. }
```

Rank.cpp

command type별 처리 진행

1.4. Read → read Data & return 할 packet 저장 & Read Latency 설정

1.4.1. packet->busPacketType = DATA

- READ command packet을 DATA packet으로 변환

1.4.2. readReturnPacket.push_back(packet)

- Data를 RL(Read Latency)만큼 기다렸다가 Data Bus로 내보낼 때까지 임시로 buffer에 저장

1.4.3. readReturnCountdown.push_back(config.RL)

- DATA packet이 bus로 나가기까지 기다려야 할 시간 (= Read Latency)을 설정

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

  2. Command Bus → receiveFromBus()

     - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case WRITE

<Data movement according to operation mode>

Index	Mode	CMD	RA13	Data Movement
①	Normal	WR	L	Data write to cell arrray
②		WR	H	Not available
③		RD	L	Data read to cell arrray
④		RD	H	Not available
⑤	FIM	WR	L	Data movement from PCU blck to cell array
⑥		WR	H	Data write to PCU register
⑦		RD	L	Data movement from cell array to PCU bock
⑧		RD	H	Not available

```
void Rank::sendToBank(BusPacket* packet)
{
    switch (packet->busPacketType)
    {
        case READ:
            if (mode_ == dramMode::SB)
                readSb(packet);
            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
                pimRank->doPIM(packet);
            else
                pimRank->readHab(packet);
            packet->busPacketType = DATA;
            readReturnPacket.push_back(packet);
            readReturnCountdown.push_back(config.RL);
            break;
        case WRITE:
            if (mode_ == dramMode::SB)
                writeSb(packet);
            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
                pimRank->doPIM(packet);
            else
                pimRank->writeHab(packet);
            delete (packet);
            break;
        case ACTIVATE:
            if (DEBUG_CMD_TRACE)
            {
                PRINTC(getModeColor(), OUTLOG_ACTIVATE("ACTIVATE"));
            }
            if (mode_ == dramMode::SB || packet->bank < 2)
            {
                PRINTC(getModeColor(), OUTLOG_ACTIVATE("ACTIVATE"));
            }
            break;
        case PRECHARGE:
            if (DEBUG_CMD_TRACE)
            {
                if (mode_ == dramMode::SB || packet->bank < 2)
                {
                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
                }
            }
            delete (packet);
            break;
```

```
void Rank::writeSb(BusPacket* packet)
{
    if (DEBUG_CMD_TRACE)
    {
        if (packet->row == config.PIM_REG_RA || pimRank->isReservedRA(packet->row))
        {
            PRINTC(GRAY, OUTLOG_ALL("WRITE"));
        }
        else
        {
            PRINT(OUTLOG_ALL("WRITE"));
        }
    }

#ifndef NO_STORAGE
    if (!(packet->row == config.PIM_REG_RA) && !pimRank->isReservedRA(packet->row))
        banks[packet->bank].write(packet);
#endif
}
```

```
void Bank::write(const BusPacket* busPacket)
{
    // TODO: move all the error checking to BusPacket so once we have a bus
    // packet,
    //           we know the fields are all legal

    if (busPacket->column >= numCols)
    {
        ERROR("== Error - Bus Packet column " << busPacket->column << " out of bounds");
        exit(-1);
    }

    // head of the list we need to search
    shared_ptr<DataStruct> rowHeadNode = rowEntries[busPacket->column];
    shared_ptr<DataStruct> foundNode = NULL;
    if ((foundNode = Bank::searchForRow(busPacket->row, rowHeadNode)) == NULL)
    {
        // not found
        shared_ptr<DataStruct> newRowNode = make_shared<DataStruct>();
        // DataStruct* newRowNode = (DataStruct*)malloc(sizeof(DataStruct));

        // insert at the head for speed
        // TODO: Optimize this data structure for speedier lookups?
        newRowNode->row = busPacket->row;
        if (busPacket->data)
            newRowNode->data = *(busPacket->data);
        newRowNode->next = rowHeadNode;
        rowEntries[busPacket->column] = newRowNode;
    }
    else
    {
        // found it, just plaster in the new data
        foundNode->data = *(busPacket->data);
        if (DEBUG_BANKS)
        {
            PRINTN(" -- Bank " << busPacket->bank << " writing to physical address 0x" << hex
                               << busPacket->physicalAddress << dec << ":");
            busPacket->printData();
            PRINT("");
        }
    }
}
```

Bank.cpp

```
unsigned inline isReservedRA(unsigned row)
{
    return (row & (1 << 13));
}
```

PIMRank.h

command type별 처리 진행

2.1. Write → SB mode 시, writeSb()(write Standard Bank) 호출

- Write을 수행하지 않는 경우
  - PIM register 접근을 위한 특정 row address(=PIM_REG_RA)
  - isResrvedRA() == true인 경우
- 일반 DRAM Memory
  - banks[]의.write() 함수 호출로 Memory 접근

```
PIM_REG_RA = 0x3fff;
PIM_ABMR_RA = 0x27ff;
PIM_SBMR_RA = 0x2fff;
```

Configuration.h

```
BurstType srf;
BurstType grfA[8];
BurstType grfB[8];
```

PIMBlock.h

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Techn

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

  2. Command Bus → receiveFromBus()

     - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case WRITE

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet
29.            }
30.            if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                    (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ &
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                    mode_ = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
51.        case PRECHARGE:
52.            if (DEBUG_CMD_TRACE)
53.            {
54.                if (mode_ == dramMode::SB || packet->bank < 2)
55.                {
56.                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                }
58.                else
59.                {
60.                    PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
```

```
1. void PIMRank::doPIM(BusPacket* packet)
2. {
3.     PIMCmd cCmd;
4.     packet->row = masked2accessibleRA(packet->row);
5.     do
6.     {
7.         cCmd.fromInt(crf.data[pimPC_]);
8.         if (DEBUG_CMD_TRACE)
9.         {
10.            PRINTC(CYAN, string((packet->busPacketType == READ) ? "READ ch" : "WRITE ch")
11.                << getChanId() << " ra" << getRankId() << " bg"
12.                << config.addrMapping.bankGroupId(packet->bank) << " b" << packet->bank
13.                << " r" << packet->row << " c" << packet->column << "|| [" << pimPC_
14.                << "] " << cCmd.toStr() << " @ " << currentClockCycle);
15.        }
16.
17.        if (cCmd.type_ == PIMCmdType::EXIT)
18.        {
19.            crfExit_ = true;
20.            break;
21.        }
22.        else if (cCmd.type_ == PIMCmdType::JUMP)
23.        {
24.            if (lastJumpIdx_ != pimPC_)
25.            {
26.                if (cCmd.loopCounter_ > 0)
27.                {
28.                    lastJumpIdx_ = pimPC_;
29.                    numJumpToBeTaken_ = cCmd.loopCounter_;
30.                }
31.            }
32.            if (numJumpToBeTaken_ > 0)
33.            {
34.                pimPC_ -= cCmd.loopOffset_;
35.                numJumpToBeTaken_--;
36.            }
37.        }
38.        else
39.        {
40.            if (cCmd.type_ == PIMCmdType::FILL || cCmd.isAuto_)
41.            {
42.                if (lastRepeatIdx_ != pimPC_)
43.                {
44.                    lastRepeatIdx_ = pimPC_;
45.                    numRepeatToBeDone_ = 8 - 1;
46.                }
47.
48.                if (numRepeatToBeDone_ > 0)
49.                {
50.                    pimPC_ -= 1;
51.                    numRepeatToBeDone_--;
52.                }
53.                else
54.                    lastRepeatIdx_ = -1;
55.            }
56.            else if (cCmd.type_ == PIMCmdType::NOP)
57.            {
58.                if (lastRepeatIdx_ != pimPC_)
59.                {
60.                    lastRepeatIdx_ = pimPC_;
61.                    numRepeatToBeDone_ = cCmd.loopCounter_;
62.                }
63.
64.                if (numRepeatToBeDone_ > 0)
65.                {
66.                    pimPC_ -= 1;
67.                    numRepeatToBeDone_--;
68.                }
69.                else
70.                    lastRepeatIdx_ = -1;
71.            }
72.
73.            for (int pimblock_id = 0; pimblock_id < config.NUM_PIM_BLOCKS; pimblock_id++)
74.            {
75.                doPIMBlock(packet, cCmd, pimblock_id);
76.
77.                if (DEBUG_PIM_BLOCK && pimblock_id == 0)
78.                {
79.                    PRINT("[BANK_R]" << packet->data->fp16ToStr());
80.                    PRINT("[CMD]" << bitset<32>(cCmd.toInt()) << "(" << cCmd.toStr() << ")");
81.                    PRINT(pimBlocks[pimblock_id].print());
82.                    PRINT("----------");
83.                }
84.            }
85.            pimPC_++;
86.            // EXIT check
87.            PIMCmd next_cmd;
88.            next_cmd.fromInt(crf.data[pimPC_]);
89.            if (next_cmd.type_ == PIMCmdType::EXIT)
90.                crfExit_ = true;
91.        } while (cCmd.type_ == PIMCmdType::JUMP);
92.    }
93. }
```

```
1.  unsigned inline masked2accessibleRA(unsigned row)
2.  {
3.      return (row & ((1 << 13) - 1));
4.  }
```

PIMRank.cpp

command type별 처리 진행

2.2. Write → PIM mode 인지 판별 → doPIM() 수행

- PPT page 30 ~ 39와 동일 동작 수행

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

• sendToBank() → case WRITE

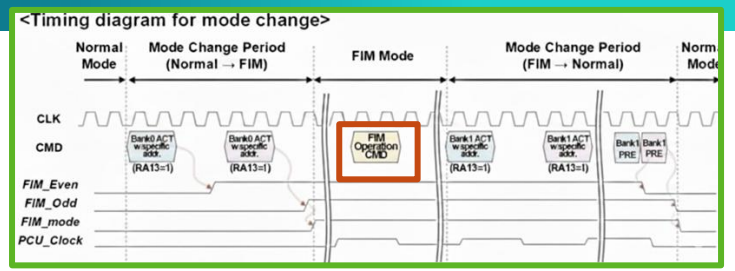
```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimR
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimR
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTI
29.            }
30.            if (mode_ == dramMode::SB && packet->row ==
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true
34.                abmr1Odd_ = (packet->bank == 1) ? true
35.                abmr2Even_ = (packet->bank == 8) ? true
36.                abmr2Odd_ = (packet->bank == 9) ? true
37.
38.                if ((config.NUM_BANKS <= 2 && abmr1Even
39.                    (config.NUM_BANKS > 2 && abmr1Even_
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_
42.                    mode_ = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB")
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
```

```
1. void PIMRank::writeHab(BusPacket* packet)
2. {
3.     if (packet->row == config.PIM_REG_RA)  // WRIO to PIM Broadcasting
4.     {
5.         if (packet->column == 0x00)
6.             controlPIM(packet);
7.         if ((0x08 <= packet->column && packet->column <= 0x0f) ||
8.             (0x18 <= packet->column && packet->column <= 0x1f))
9.         {
10.            if (DEBUG_CMD_TRACE)
11.            {
12.                if (packet->column - 8 < 8)
13.                    PRINTC(GREEN, OUTLOG_B_GRF_A("BWRITE_GRF_A"));
14.                else
15.                    PRINTC(GR
16.            }
17. #ifndef NO_STORAGE
18.            for (int pb = 0;
19.            {
20.                if (packet->
21.                    pimBlocks
22.                else
23.                    pimBlocks
24.            }
25. #endif
26.        }
27.        else if (0x04 <= pa
28.        {
29.            if (DEBUG_CMD_TR
30.                PRINTC(GREEN
31.            crf.bst[packet->
32.        }
33.        else if (packet->co
34.        {
35.            if (DEBUG_CMD_TR
36.                PRINTC(GREEN
37.            for (int pb = 0;
38.        }
39.    }
40.    else if (isReservedRA(p
41.    {
42.        PRINTC(GRAY, OUTLOG_
43.    }
44.    else  // PIM (only GRF)
45.    {
46.        PRINTC(GREEN, OUTLO
47.
48. #ifndef NO_STORAGE
49.        int grf_id = getGrf
50.        for (int pb = 0; pb
51.        {
52.            if (packet->bank
53.            {
54.                *(packet->da
55.                rank->banks[
56.            }
57.            else if (packet-
58.            {
59.                *(packet->da
60.                rank->banks[
61.            }
62.        }
63. #endif
64.    }
65. }
```

```
1. void PIMRank::controlPIM(BusPacket* packet)
2. {
3.     uint8_t grf_a_zeroize = packet->data->u8Data_[20];
4.     if (grf_a_zeroize)
5.     {
6.         if (DEBUG_CMD_TRACE)
7.         {
8.             PRINTC(RED, OUTLOG_CH_RA("GRF_A_ZEROIZE"));
9.         }
10.        BurstType burst_zero;
11.        for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
12.        {
13.            for (int i = 0; i < 8; i++) pimBlocks[pb].grfA[i] = burst_zero;
14.        }
15.    }
16.    uint8_t grf_b_zeroize = packet->data->u8Data_[21];
17.    if (grf_b_zeroize)
18.    {
19.        if (DEBUG_CMD_TRACE)
20.        {
21.            PRINTC(RED, OUTLOG_CH_RA("GRF_B_ZEROIZE"));
22.        }
23.        BurstType burst_zero;
24.        for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
25.        {
26.            for (int i = 0; i < 8; i++) pimBlocks[pb].grfB[i] = burst_zero;
27.        }
28.    }
29.    pimOpMode_ = packet->data->u8Data_[0] & 1;
30.    toggleEvenBank_ = !(packet->data->u8Data_[16] & 1);
31.    toggleOddBank_ = !(packet->data->u8Data_[16] & 2);
32.    toggleRa13h_ = (packet->data->u8Data_[16] & 4);
33.
34.    if (pimOpMode_)
35.    {
36.        rank->mode_ = dramMode::HAB_PIM;
37.        pimPC_ = 0;
38.        lastJumpIdx_ = numJumpToBeTaken_ = lastRepeatIdx_ = numRepeatToBeDone_ = -1;
39.        crfExit_ = false;
40.        PRINTC(RED, OUTLOG_CH_RA("HAB_PIM"));
41.    }
42.    else
43.    {
44.        rank->mode_ = dramMode::HAB;
45.        PRINTC(RED, OUTLOG_CH_RA("HAB mode"));
46.    }
47. }
```

PIMRank.cpp

PIMRank.cpp



command type별 처리 진행

2.3. Write → PIM mode 인지 판별 → 아닐 시 → writeHab() 수행

writeHab() → controlPIM()

- GRF 초기화
  - u8Data_[20]: GRF_A Reg를 0으로 초기화하는 flag
  - u8Data_[21]: GRF_B Reg를 0으로 초기화하는 flag
- PIM trigger bit 설정
  - pimOpMode_ = packet->data->u8Data_[0] & 1
    - 1이면 PIM 연산 mode
    - 0이면 HAB mode
  - toggleEven/OddBank_ → isToggleCond()에서 사용
    - 각각 even/odd Bank에 대한 접근 시 doPIM() 을 호출할지 여부를 결정
  - toggleRa13h_ → isToggleCond()에서 사용
    - doPIM() 호출 시 isReservedRA 여부를 고려할 지를 결정
- PIM 동작 mode 전환 (HAB ↔ HAB_PIM)

Reference: Hardware_Architecture_ar nology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

2. Command Bus → receiveFromBus()

• sendToBank

case WRITE

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimR
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimR
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTI
29.            }
30.            if (mode_ == dramMode::SB && packet->row ==
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true
34.                abmr1Odd_ = (packet->bank == 1) ? true
35.                abmr2Even_ = (packet->bank == 8) ? true
36.                abmr2Odd_ = (packet->bank == 9) ? true
37.
38.                if ((config.NUM_BANKS <= 2 && abmr1Even
39.                    (config.NUM_BANKS > 2 && abmr1Even_
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_
42.                    mode_ = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB")
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
```

```
1. void PIMRank::writeHab(BusPacket* packet)
2. {
3.     if (packet->row == config.PIM_REG_RA)  // WRIO to PIM Broadcasting
4.     {
5.         if (packet->column == 0x00)
6.             controlPIM(packet);
7.         if ((0x08 <= packet->column && packet->column <= 0x0f) ||
8.             (0x18 <= packet->column && packet->column <= 0x1f))
9.         {
10.            if (DEBUG_CMD_TRACE)
11.            {
12.                if (packet->column - 8 < 8)
13.                    PRINTC(GREEN, OUTLOG_B_GRF_A("BWRITE_GRF_A"));
14.                else
15.                    PRINTC(GREEN, OUTLOG_B_GRF_B("BWRITE_GRF_B"));
16.            }
17. #ifndef NO_STORAGE
18.            for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
19.            {
20.                if (packet->column - 8 < 8)
21.                    pimBlocks[pb].grfA[packet->column - 0x8] = *(packet->data);
22.                else
23.                    pimBlocks[pb].grfB[packet->column - 0x18] = *(packet->data);
24.            }
25. #endif
26.        }
27.        else if (0x04 <= packet->column && packet->column <= 0x07)
28.        {
29.            if (DEBUG_CMD_TRACE)
30.                PRINTC(GREEN, OUTLOG_B_CRF("BWRITE_CRF"));
31.            crf.bst[packet->column - 0x04] = *(packet->data);
32.        }
33.        else if (packet->column == 0x1)
34.        {
35.            if (DEBUG_CMD_TRACE)
36.                PRINTC(GREEN, OUTLOG_CH_RA("BWRITE_SRF"));
37.            for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++) pimBlocks[pb].srf = *(packet->data);
38.        }
39.    }
40.    else if (isReservedRA(packet->row))
41.    {
42.        PRINTC(GRAY, OUTLOG_ALL("WRITE"));
43.    }
44.    else  // PIM (only GRF) to Bank Move
45.    {
46.        PRINTC(GREEN, OUTLOG_ALL("PIM_TO_BANK"));
47.
48. #ifndef NO_STORAGE
49.        int grf_id = getGrfIdx(packet->column);
50.        for (int pb = 0; pb < config.NUM_PIM_BLOCKS; pb++)
51.        {
52.            if (packet->bank == 0)
53.            {
54.                *(packet->data) = pimBlocks[pb].grfA[grf_id];
55.                rank->banks[pb * 2].write(packet);  // basically read from bank;
56.            }
57.            else if (packet->bank == 1)
58.            {
59.                *(packet->data) = pimBlocks[pb].grfB[grf_id];
60.                rank->banks[pb * 2 + 1].write(packet);  // basically read from bank.
61.            }
62.        }
63. #endif
64.    }
65. }
```

PIMRank.cpp

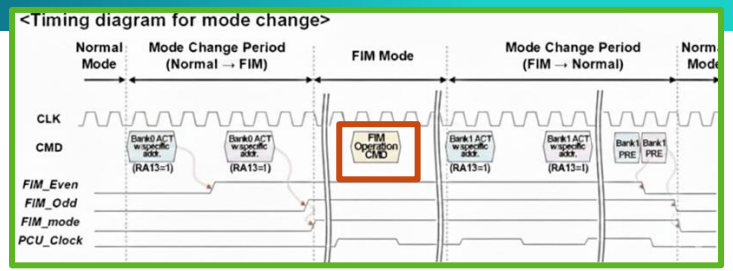
```
PIM_REG_RA = 0x3fff;
PIM_ABMR_RA = 0x27ff;
PIM_SBMR_RA = 0x2fff;
```

Configuration.h

```
BurstType srf;
BurstType grfA[8];
BurstType grfB[8];
```

PIMBlock.h

Rank.cpp



command type별 처리 진행

2.3. Write → PIM mode 인지 판별 → 아닐 시 → writeHab() 수행

writeHab()

- Row Address가 PIM_REG_RA와 동일 시
  - column == 0x00
    - controlPIM() 함수를 호출
  - column 0x08 ~ 0x0f || 0x18 ~ 0x1f
    - 모든 PIM Block의 GRF에 동일 data write
  - column 0x04 ~ 0x07
    - CRF(command register file)에 data write
  - column 0x1
    - 모든 PIM 블록의 SRF(scalar register file)에 동일 data write
- isReservedRA()==true 시
  - Write를 skip
- 그 외
  - GRF Register에서 even/odd DRAM Bank로 저장

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

  2. Command Bus → receiveFromBus()

     - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case ACTIVATE

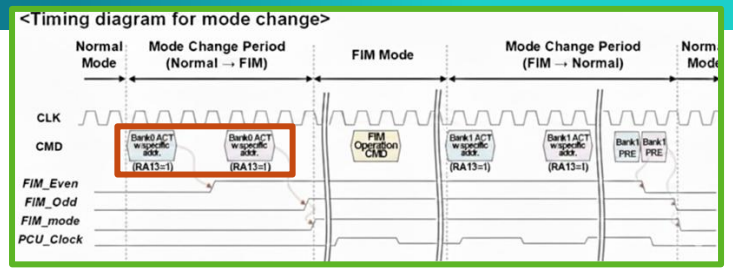
```
1.  void Rank::sendToBank(BusPacket* packet)
2.  {
3.      switch (packet->busPacketType)
4.      {
5.          case READ:
6.              if (mode_ == dramMode::SB)
7.                  readSb(packet);
8.              else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                  pimRank->doPIM(packet);
10.             else
11.                 pimRank->readHab(packet);
12.             packet->busPacketType = DATA;
13.             readReturnPacket.push_back(packet);
14.             readReturnCountdown.push_back(config.RL);
15.             break;
16.         case WRITE:
17.             if (mode_ == dramMode::SB)
18.                 writeSb(packet);
19.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                 pimRank->doPIM(packet);
21.             else
22.                 pimRank->writeHab(packet);
23.             delete (packet);
24.             break;
25.         case ACTIVATE:
26.             if (DEBUG_CMD_TRACE)
27.             {
28.                 PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.             }
30.             if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                 packet->column == 0x1f)
32.             {
33.                 abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                 abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                 abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                 abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                 if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                     (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                 {
41.                     abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                     mode_ = dramMode::HAB;
43.                     if (DEBUG_CMD_TRACE)
44.                     {
45.                         PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                     }
47.                 }
48.             }
49.             delete (packet);
50.             break;
51.         case PRECHARGE:
52.             if (DEBUG_CMD_TRACE)
53.             {
54.                 if (mode_ == dramMode::SB || packet->bank < 2)
55.                 {
56.                     PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                 }
58.                 else
59.                 {
60.                     PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                 }
62.             }
63.
64.             if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
65.             {
66.                 sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
67.                 sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
68.
69.                 if (sbmr1_ && sbmr2_)
70.                 {
71.                     sbmr1_ = sbmr2_ = false;
72.                     mode_ = dramMode::SB;
73.                     if (DEBUG_CMD_TRACE)
74.                     {
75.                         PRINTC(RED, OUTLOG_CH_RA("SB mode"));
76.                     }
77.                 }
78.             }
79.
80.             delete (packet);
81.             break;
82.
83.         case REF:
84.             refreshWaiting = false;
85.             if (DEBUG_CMD_TRACE)
86.             {
87.                 PRINT(OUTLOG_CH_RA("REF"));
88.             }
89.             delete (packet);
90.             break;
91.
92.         case DATA:
93.             delete (packet);
94.             break;
95.
96.         default:
97.             ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
98.             exit(0);
99.             break;
100.    }
101. }
```

ABMR: All-Bank Mode Request

```
PIM_REG_RA  = 0x3fff;
PIM_ABMR_RA = 0x27ff;
PIM_SBMR_RA = 0x2fff;
```

Configuration.h

Rank.cpp



command type별 처리 진행

2.3. ACTIVATE

- 일반 ACTIVATE 동작은 Rank::updateBank( )에서 수행 (PPT page 22참고)
- Rank::sendToBank()의 ACTIVATE에서는 Mode Switching 수행
  - Rank가 현재 SB mode

    & row address == PIM_ABMR_RA(=mode 변경을 위한 address)

    & column address == 0x1f(=ABMR의 reserved Row Address)
    - packet의 target bank가 0이면, abmr1Even_ = true
    - packet의 target bank가 1이면, abmr1Odd_ = true
    - packet의 target bank가 8이면, abmr2Even_ = true
    - packet의 target bank가 9이면, abmr2Odd_ = true
    - 즉, 서로 다른 Bank들에 모두 Mode 전환 ACTIVATE가 선행되어야 HAB mode로 switching 가능
    - HAB mode switch 후, 다음 mode switch을 위해 모든 flag를 다시 false로 초기화 및 현재 packet delete
    - NUM_BANKS가 2개 이하이면, abmr1Even_과 abmr1Odd_만 check함

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

  2. Command Bus → receiveFromBus()

     - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case PRECHARGE

```
1.  void Rank::sendToBank(BusPacket* packet)
2.  {
3.      switch (packet->busPacketType)
4.      {
5.          case READ:
6.              if (mode_ == dramMode::SB)
7.                  readSb(packet);
8.              else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                  pimRank->doPIM(packet);
10.             else
11.                 pimRank->readHab(packet);
12.             packet->busPacketType = DATA;
13.             readReturnPacket.push_back(packet);
14.             readReturnCountdown.push_back(config.RL);
15.             break;
16.         case WRITE:
17.             if (mode_ == dramMode::SB)
18.                 writeSb(packet);
19.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                 pimRank->doPIM(packet);
21.             else
22.                 pimRank->writeHab(packet);
23.             delete (packet);
24.             break;
25.         case ACTIVATE:
26.             if (DEBUG_CMD_TRACE)
27.             {
28.                 PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.             }
30.             if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                 packet->column == 0x1f)
32.             {
33.                 abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                 abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                 abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                 abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                 if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                     (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                 {
41.                     abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                     mode_ = dramMode::HAB;
43.                     if (DEBUG_CMD_TRACE)
44.                     {
45.                         PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                     }
47.                 }
48.             }
49.             delete (packet);
50.             break;
```

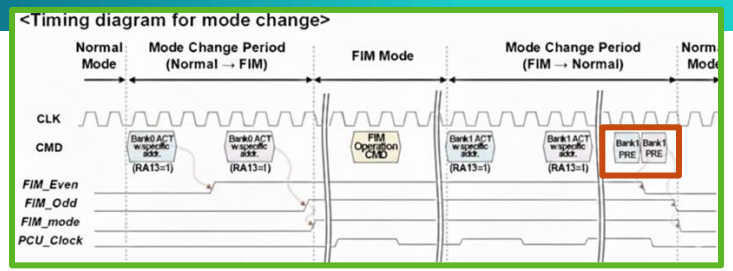
SBMR: Single-Bank Mode Request

```
PIM_REG_RA = 0x3fff;
PIM_ABMR_RA = 0x27ff;
PIM_SBMR_RA = 0x2fff;
```

Configuration.h

```
51.         case PRECHARGE:
52.             if (DEBUG_CMD_TRACE)
53.             {
54.                 if (mode_ == dramMode::SB || packet->bank < 2)
55.                 {
56.                     PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                 }
58.                 else
59.                 {
60.                     PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                 }
62.             }
63.
64.             if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
65.             {
66.                 sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
67.                 sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
68.
69.                 if (sbmr1_ && sbmr2_)
70.                 {
71.                     sbmr1_ = sbmr2_ = false;
72.                     mode_ = dramMode::SB;
73.                     if (DEBUG_CMD_TRACE)
74.                     {
75.                         PRINTC(RED, OUTLOG_CH_RA("SB mode"));
76.                     }
77.                 }
78.             }
79.
80.             delete (packet);
81.             break;
82.
83.         case REF:
84.             refreshWaiting = false;
85.             if (DEBUG_CMD_TRACE)
86.             {
87.                 PRINT(OUTLOG_CH_RA("REF"));
88.             }
89.             delete (packet);
90.             break;
91.
92.         case DATA:
93.             delete (packet);
94.             break;
95.
96.         default:
97.             ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
98.             exit(0);
99.             break;
100.    }
101. }
```

Rank.cpp



command type별 처리 진행

2.4. PRECHARGE

- 일반 PRECHARGE 동작은 Rank::updateBank( )에서 수행
  (PPT page 22참고)
- Rank::sendToBank()의 PRECHARGE에서는 Mode Switching 수행
  - Rank가 현재 HAB mode
    & row address == PIM_SBMR_RA(=mode 변경을 위한 address)
    - packet의 target bank가 0이면, sbmr1_ = true
    - packet의 target bank가 1이면, sbmr2_ = true
    - 즉, 서로 다른 Bank들에 모두 Mode 전환 PRECHARGE가 선행되어야 HAB mode로 switching 가능
    - SB mode switch 후, 다음 mode switch을 위해 모든 flag를 다시 false로 초기화 및 현재 packet delete
- *PRECHARGE를 이용한 SB mode 복귀 절차가 더 간단한 이유*
  - *system이 이미 PIM이라는 특수 목적의 작업을 수행 중인 신뢰할 수 있는 상태에 있기 때문임 (PIM device driver에 의해 제어됨)*
  - *일반 DRAM(=SB mode)에서는 command를 예측 불가능*

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

  2. Command Bus → receiveFromBus()

     - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case REF

```
1. void Rank::sendToBank(BusPacket* packet)
2. {
3.     switch (packet->busPacketType)
4.     {
5.         case READ:
6.             if (mode_ == dramMode::SB)
7.                 readSb(packet);
8.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                 pimRank->doPIM(packet);
10.            else
11.                pimRank->readHab(packet);
12.            packet->busPacketType = DATA;
13.            readReturnPacket.push_back(packet);
14.            readReturnCountdown.push_back(config.RL);
15.            break;
16.        case WRITE:
17.            if (mode_ == dramMode::SB)
18.                writeSb(packet);
19.            else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                pimRank->doPIM(packet);
21.            else
22.                pimRank->writeHab(packet);
23.            delete (packet);
24.            break;
25.        case ACTIVATE:
26.            if (DEBUG_CMD_TRACE)
27.            {
28.                PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.            }
30.            if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                packet->column == 0x1f)
32.            {
33.                abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                    (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                {
41.                    abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                    mode_ = dramMode::HAB;
43.                    if (DEBUG_CMD_TRACE)
44.                    {
45.                        PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                    }
47.                }
48.            }
49.            delete (packet);
50.            break;
51.        case PRECHARGE:
52.            if (DEBUG_CMD_TRACE)
53.            {
54.                if (mode_ == dramMode::SB || packet->bank < 2)
55.                {
56.                    PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                }
58.                else
59.                {
60.                    PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                }
62.            }
63.
64.            if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
65.            {
66.                sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
67.                sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
68.
69.                if (sbmr1_ && sbmr2_)
70.                {
71.                    sbmr1_ = sbmr2_ = false;
72.                    mode_ = dramMode::SB;
73.                    if (DEBUG_CMD_TRACE)
74.                    {
75.                        PRINTC(RED, OUTLOG_CH_RA("SB mode"));
76.                    }
77.                }
78.            }
79.
80.            delete (packet);
81.            break;
82.
83.        case REF:
84.            refreshWaiting = false;
85.            if (DEBUG_CMD_TRACE)
86.            {
87.                PRINT(OUTLOG_CH_RA("REF"));
88.            }
89.            delete (packet);
90.            break;
91.
92.        case DATA:
93.            delete (packet);
94.            break;
95.
96.        default:
97.            ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
98.            exit(0);
99.            break;
100.    }
101. }
```

Rank.cpp

command type별 처리 진행

2.5. REF

- Rank가 REFRESH command를 성공적으로 처리했음을 알리고, 관련 state를 정리하는 작업을 수행
- REF command의 흐름은 다음과 같다
  1. Memory Controller가 Refresh의 필요성을 확인 (counter 이용)
  2. Refresh가 필요한 Rank에 Refresh command를 issue
  3. Issue된 Refresh command는 Rank까지 bus를 이용하여 이동 (tCMD만큼의 latency 발생)
  4. Rank에 Refresh command가 도착하고 Refresh 동작 수행
- Rank->refreshWaiting = true
  - 해당 Rank가 ref cmd packet을 bus로부터 받는 것을 기다리는 중
- sendToBank()에서 REF command를 Rank에서 처리하는 시점
  - refreshWaiting = false로 되돌림
  - Rank가 기다리던 REF command를 받았음을 알려줌
- 해당 Rank가 Refresh동안 다른 command를 issue하지 못하도록 하는 역할은 MemoryController::updateCommandQueue()에서 수행

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → . . . → Rank::receiveFromBus() → Rank::sendToBank()

  2. Command Bus → receiveFromBus()

     - sendToBank(): Command가 실제 Bank 또는 PIM에서 실행 → case DATA

```
1.  void Rank::sendToBank(BusPacket* packet)
2.  {
3.      switch (packet->busPacketType)
4.      {
5.          case READ:
6.              if (mode_ == dramMode::SB)
7.                  readSb(packet);
8.              else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
9.                  pimRank->doPIM(packet);
10.             else
11.                 pimRank->readHab(packet);
12.             packet->busPacketType = DATA;
13.             readReturnPacket.push_back(packet);
14.             readReturnCountdown.push_back(config.RL);
15.             break;
16.         case WRITE:
17.             if (mode_ == dramMode::SB)
18.                 writeSb(packet);
19.             else if (mode_ == dramMode::HAB_PIM && pimRank->isToggleCond(packet))
20.                 pimRank->doPIM(packet);
21.             else
22.                 pimRank->writeHab(packet);
23.             delete (packet);
24.             break;
25.         case ACTIVATE:
26.             if (DEBUG_CMD_TRACE)
27.             {
28.                 PRINTC(getModeColor(), OUTLOG_ALL("ACTIVATE") << " tag : " << packet->tag);
29.             }
30.             if (mode_ == dramMode::SB && packet->row == config.PIM_ABMR_RA &&
31.                 packet->column == 0x1f)
32.             {
33.                 abmr1Even_ = (packet->bank == 0) ? true : abmr1Even_;
34.                 abmr1Odd_ = (packet->bank == 1) ? true : abmr1Odd_;
35.                 abmr2Even_ = (packet->bank == 8) ? true : abmr2Even_;
36.                 abmr2Odd_ = (packet->bank == 9) ? true : abmr2Odd_;
37.
38.                 if ((config.NUM_BANKS <= 2 && abmr1Even_ && abmr1Odd_) ||
39.                     (config.NUM_BANKS > 2 && abmr1Even_ && abmr1Odd_ && abmr2Even_ && abmr2Odd_))
40.                 {
41.                     abmr1Even_ = abmr1Odd_ = abmr2Even_ = abmr2Odd_ = false;
42.                     mode_ = dramMode::HAB;
43.                     if (DEBUG_CMD_TRACE)
44.                     {
45.                         PRINTC(RED, OUTLOG_CH_RA("HAB") << " tag : " << packet->tag);
46.                     }
47.                 }
48.             }
49.             delete (packet);
50.             break;
51.         case PRECHARGE:
52.             if (DEBUG_CMD_TRACE)
53.             {
54.                 if (mode_ == dramMode::SB || packet->bank < 2)
55.                 {
56.                     PRINTC(getModeColor(), OUTLOG_PRECHARGE("PRECHARGE"));
57.                 }
58.                 else
59.                 {
60.                     PRINTC(GRAY, OUTLOG_PRECHARGE("PRECHARGE"));
61.                 }
62.             }
63.
64.             if (mode_ == dramMode::HAB && packet->row == config.PIM_SBMR_RA)
65.             {
66.                 sbmr1_ = (packet->bank == 0) ? true : sbmr1_;
67.                 sbmr2_ = (packet->bank == 1) ? true : sbmr2_;
68.
69.                 if (sbmr1_ && sbmr2_)
70.                 {
71.                     sbmr1_ = sbmr2_ = false;
72.                     mode_ = dramMode::SB;
73.                     if (DEBUG_CMD_TRACE)
74.                     {
75.                         PRINTC(RED, OUTLOG_CH_RA("SB mode"));
76.                     }
77.                 }
78.             }
79.
80.             delete (packet);
81.             break;
82.
83.         case REF:
84.             refreshWaiting = false;
85.             if (DEBUG_CMD_TRACE)
86.             {
87.                 PRINT(OUTLOG_CH_RA("REF"));
88.             }
89.             delete (packet);
90.             break;
91.
92.         case DATA:
93.             delete (packet);
94.             break;
95.
96.         default:
97.             ERROR("== Error - Unknown BusPacketType trying to be sent to Bank");
98.             exit(0);
99.             break;
100.    }
101. }
```

Rank.cpp

command type별 처리 진행

2.6. DATA

- WRITE 시, Data Bus를 통해 전달된 DATA packet의 임무가 끝났음을 확인하고 delete
- WRITE command의 흐름은 다음과 같다
  1. CPU 등에서 Write Transaction이 들어옴
  2. WIRTE command packet이 MC의 CommandQueue에 들어감
  3. WIRTE command packet은 CommandQueue의 Scheduling에 따라 적절한 때에 MC(=MemoryController)에 전달됨
  4. MemoryController는 이 WRITE command를 Command Bus를 통해 Rank로 transfer
  5. Rank::receiveFromBus()가 WRITE packet을 수신하고, sendToBank()를 호출
  6. writeSb() 또는 pimRank->writeHab() 함수가 호출됨. 최종적으로 banks[packet->bank].write(packet)가 실행
  7. MemoryController는 WRITE command를 issue 후, WL(Write Latency) cycle만큼 기다림
  8. WL cycle이 지나면, MemoryController는 WRITE에 해당하는 DATA packet을 Data Bus에 issue (이 DATA는 BL/2 cycle 동안 Data Bus를 점유)
  9. BL/2 cycle이 지나 DATA packet이 Rank에 도착 시, Rank::receiveFromBus()가 DATA를 수신 및 sendToBank() 호출 → delete (packet) 실행 됨

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

[cf] READ 시의 DATA는 RANK → MC
WRITE 시의 DATA는 MC → RANK

Samsung PIM Simulator hbm read test flow

❑ hbm read test flow – measureCycle()

- measureCycle() → MultiChannelMemorySystem::actual_update() → MemorySystem::update() → MemoryController::update()

  3. Write Data Bus

  - Host(MC)에서 DRAM(Rank)으로 가는 Write Data 흐름을 관리
  - dataCyclesLeft: data Burst가 유지되는 시간을 추적 → WriteDataDone: counter가 0이 되면, MC는 bus transfer가 끝났음을 MemorySystem에 알림
  - Transaction 관리: numOnTheFlyTransactions를 줄여서, 현재 처리 중인 작업 하나가 끝났음을 기록

```
1.  void MemoryController::update()
2.  {
3.      updateBankState();
4.      // check for outgoing command packets and handle countdowns
5.      if (outgoingCmdPacket != NULL)
6.      {
7.          cmdCyclesLeft--;
8.          if (cmdCyclesLeft == 0)  // packet is ready to be received by rank
9.          {
10.             (*ranks)[outgoingCmdPacket->rank]->receiveFromBus(outgoingCmdPacket);
11.             outgoingCmdPacket = NULL;
12.         }
13.     }
14.
15.     // check for outgoing data packets and handle countdowns
16.     if (outgoingDataPacket != NULL)
17.     {
18.         dataCyclesLeft--;
19.         if (dataCyclesLeft == 0)
20.         {
21.             // inform upper levels that a write is done
22.             if (parentMemorySystem->WriteDataDone != NULL)
23.             {
24.                 (*parentMemorySystem->WriteDataDone)(parentMemorySystem->systemID,
25.                                                      outgoingDataPacket->physicalAddress,
26.                                                      currentClockCycle);
27.             }
28.             parentMemorySystem->numOnTheFlyTransactions--;
29.             (*ranks)[outgoingDataPacket->rank]->receiveFromBus(outgoingDataPacket);
30.             outgoingDataPacket = NULL;
31.         }
32.     }
33.
```

[cf] packet 구성

```
cmdPacket
패킷 구성:
      busPacketType: ACTIVATE
      physicalAddress: 0x12345600
      rank: 0
      bank: 2
      row: 1024
      column: (사용 안 함)
      data: NULL

WRITE 를 위한 DataPacket
패킷 구성:
      busPacketType: DATA
      physicalAddress: 0x12345600
      rank: 0
      bank: 2
      row: 1024
      column: 8
      data: [0xAF, 0xBE, 0xAD, ...] (64 바이트 분량의 실제 값) or NULL
```

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator gemv test flow

❑ gemv test flow

- Samsung PIM Simulator는 GoogleTest(gtest)기반의 simulation test를 수행함
- 따라서, 이 flow를 Samsung PIM Simulator가 gemv operation을 어떻게 test하는지 따라가며 알아본다

./sim --gtest_filter=PIMKernelFixture.gemv

```cpp
#include <bitset>
#include <cstdint>
#include <iostream>
#include <limits>
#include <memory>
#include <random>

#include "Burst.h"
#include "MultiChannelMemorySystem.h"
#include "gtest/gtest.h"
#include "tests/PIMKernel.h"
#include "tests/TestCases.h"

using namespace std;

int main(int argc, char* argv[])
{
    if (argc > 1)
    {
        ::testing::InitGoogleTest(&argc, argv);
    }
    else
    {
        ::testing::InitGoogleTest(&argc, argv);
        testing::GTEST_FLAG(filter) = "-*bw*.*";
    }

    return RUN_ALL_TESTS();
}
```

MainTest.cpp

```cpp
TEST_F(PIMKernelFixture, gemv)
{
    shared_ptr<PIMKernel> kernel = make_pim_kernel();

    uint32_t batch_size = 1;
    uint32_t output_dim = 4096;
    uint32_t input_dim = 1024;

    DataDim *dim_data = new DataDim(KernelType::GEMV, batch_size, output_dim, input_dim, true);
    dim_data->printDim(KernelType::GEMV);

    reduced_result_ = new BurstType[dim_data->dimTobShape(output_dim)];
    result_ = getResultPIM(KernelType::GEMV, dim_data, kernel, result_);

    testStatsClear();
    expectAccuracy(KernelType::GEMV, output_dim, dim_data->output_npbst_,
                   dim_data->getNumElementsPerBlocks());

    delete[] result_;
    delete[] reduced_result_;
    delete dim_data;
}
```

KernelTestCases.cpp

Reference: Hardware_Architecture_and_Software_Stack_for_PIM_Based_on_Commercial_DRAM_Technology_Industrial_Product

Samsung PIM Simulator gemv test flow

❑ gemv test flow

- Samsung PIM Simulator는 GoogleTest(gtest)기반의 simulation test를 수행함
- 따라서, 이 flow를 Samsung PIM Simulator가 gemv operation을 어떻게 test하는지 따라가며 알아본다

```
1. TEST_F(PIMKernelFixture, gemv)
2. {
3.     shared_ptr<PIMKernel> kernel = make_pim_kernel();
4.
5.     uint32_t batch_size = 1;
6.     uint32_t output_dim = 4096;
7.     uint32_t input_dim = 1024;
8.
9.     DataDim *dim_data = new DataDim(KernelType::GEMV, batch_size, output_dim, input_dim, true);
10.    dim_data->printDim(KernelType::GEMV);
11.
12.    reduced_result_ = new BurstType[dim_data->dimTobShape(output_dim)];
13.    result_ = getResultPIM(KernelType::GEMV, dim_data, kernel, result_);
14.
15.    testStatsClear();
16.    expectAccuracy(KernelType::GEMV, output_dim, dim_data->output_npbst_,
17.                   dim_data->getNumElementsPerBlocks());
18.
19.    delete[] result_;
20.    delete[] reduced_result_;
21.    delete dim_data;
22. }
```

KernelTestCases.cpp

```
1.    shared_ptr<PIMKernel> make_pim_kernel()
2.    {
3.        shared_ptr<MultiChannelMemorySystem> mem = make_shared<MultiChannelMemorySystem>(
4.            "ini/HBM2_samsung_2M_16B_x64.ini", "system_hbm_64ch.ini", ".", "example_app",
5.            256 * 64 * 2);
6.        int numPIMChan = 64;
7.        int numPIMRank = 1;
8.        shared_ptr<PIMKernel> kernel = make_shared<PIMKernel>(mem, numPIMChan, numPIMRank);
9.
10.       return kernel;
11.   }
```